



LatticeMico32 Processor Reference Manual

Lattice Semiconductor Corporation
5555 NE Moore Court
Hillsboro, OR 97124
(503) 268-8000

September 22, 2006

Copyright

Copyright © 2006 Lattice Semiconductor Corporation.

This document may not, in whole or part, be copied, photocopied, reproduced, translated, or reduced to any electronic medium or machine-readable form without prior written consent from Lattice Semiconductor Corporation.

Trademarks

Lattice Semiconductor Corporation, L Lattice Semiconductor Corporation (logo), L (stylized), L (design), Lattice (design), LSC, E²CMOS, Extreme Performance, FlexiMAC, FlexiPCS, GAL, GDX, Generic Array Logic, HDL Explorer, IPexpress, ISP, ispATE, ispCLOCK, ispDOWNLOAD, ispGAL, ispGDS, ispGDX, ispGDXV, ispGDX2, ispGENERATOR, ispJTAG, ispLEVER, ispLEVERCORE, ispLSI, ispMACH, ispPAC, ispTRACY, ispTURBO, ispVIRTUAL MACHINE, ispVM, ispXP, ispXPGA, ispXPLD, LatticeEC, LatticeECP2, LatticeECP2M, LatticeECP, LatticeECP-DSP, LatticeMico8, LatticeMico32, LatticeSC, LatticeXP, MACH, MachXO, MACO, ORCA, PAC, PAC-Designer, PAL, Performance Analyst, PureSPEED, Silicon Forest, Speedlocked, Speed Locking, SuperBIG, SuperCOOL, SuperFAST, SuperWIDE, sysCLOCK, sysCONFIG, sysDSP, sysHSI, sysI/O, sysMEM, The Simple Machine for Complex Design, TransFR, UltraMOS, and specific product designations are either registered trademarks or trademarks of Lattice Semiconductor Corporation or its subsidiaries in the United States and/or other countries. ISP and Bringing the Best Together are service marks of Lattice Semiconductor Corporation.

Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

Disclaimers

NO WARRANTIES: THE INFORMATION PROVIDED IN THIS DOCUMENT IS “AS IS” WITHOUT ANY EXPRESS OR IMPLIED WARRANTY OF ANY KIND INCLUDING WARRANTIES OF ACCURACY, COMPLETENESS, MERCHANTABILITY, NONINFRINGEMENT OF INTELLECTUAL PROPERTY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL LATTICE SEMICONDUCTOR CORPORATION (LSC) OR ITS SUPPLIERS BE LIABLE FOR ANY DAMAGES WHATSOEVER (WHETHER DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL, INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF PROFITS, BUSINESS INTERRUPTION, OR LOSS OF INFORMATION) ARISING OUT OF THE USE OF OR INABILITY TO USE THE INFORMATION PROVIDED ON THIS SITE, EVEN IF LSC HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. BECAUSE SOME JURISDICTIONS PROHIBIT THE EXCLUSION OR LIMITATION OF CERTAIN LIABILITY, SOME OF THE ABOVE LIMITATIONS MAY NOT APPLY TO YOU.

LSC may make changes to these materials, specifications, or information, or to the products described herein, at any time without notice. LSC makes no commitment to update this documentation. LSC reserves the right to discontinue any product or service without notice and assumes no obligation to correct any errors contained herein or to advise any user of this document of any correction if such be made. LSC recommends its customers obtain the

latest version of the relevant information to establish, before ordering, that the information being relied upon is current.

Type Conventions Used in This Document

Convention	Meaning or Use
Bold	Items in the user interface that you select or click. Text that you type into the user interface.
<i><Italic></i>	Variables in commands, code syntax, and path names.
Ctrl+L	Press the two keys at the same time.
Courier	Code examples. Messages, reports, and prompts from the software.
...	Omitted material in a line of code.
.	Omitted lines in code and report examples.
[]	Optional items in syntax descriptions. In bus specifications, the brackets are required.
()	Grouped items in syntax descriptions.
{ }	Repeatable items in syntax descriptions.
	A choice between items in syntax descriptions.

Contents

Chapter 1	LatticeMico32 Processor and Systems	1
Chapter 2	Programmer's Model	5
	Pipeline Architecture	5
	Data Types	6
	Register Architecture	7
	General Purpose Registers	7
	Control and Status Registers	9
	Memory Architecture	13
	Address Space	13
	Endianness	13
	Address Alignment	14
	Stack Layout	14
	Caches	15
	Exceptions	17
	Exception Processing	17
	Exception Handlers	18
	Nested Exceptions	22
	Remapping the Exception Table	22
	Reset Summary	23
	Using Breakpoints	23
	Using Watchpoints	24
	Debug Architecture	24
	DC – Debug Control	25
	DEBA – Debug Exception Base Address	25
	JTX – JTAG UART Transmit Register	26
	JRX – JTAG UART Receive Register	26
	BPn – Breakpoint	26
	WPn – Watchpoint	27
	Instruction Set Categories	27
	Arithmetic	27

	Logic	27
	Comparison	28
	Shift	28
	Data Transfer	28
	Program Flow Control	29
Chapter 3	Configuring the LatticeMico32 Processor	31
	Configuration Options	31
	EBR Use	34
Chapter 4	WISHBONE Interconnect Architecture	35
	Introduction to WISHBONE Interconnect	35
	WISHBONE Registered Feedback Mode	36
	CTI_IO()	36
	BTE_IO()	37
	Component Signals	38
	Master Port/Signal Descriptions	39
	Slave Port/Signal Descriptions	40
Chapter 5	Instruction Set	43
	Instruction Formats	43
	Opcode Look-Up Table	44
	Pseudo-Instructions	45
	Instruction Descriptions	45
	add	45
	addi	46
	and	46
	andhi	47
	andi	47
	b	48
	be	48
	bg	49
	bge	49
	bgeu	50
	bgu	50
	bi	51
	bne	51
	break	52
	bret	52
	call	53
	calli	53
	cmpe	54
	cmpei	54
	cmpg	55
	cmpgi	55
	cmpge	56
	cmpgei	56
	cmpgeu	57
	cmpgeui	57
	cmpgu	58
	cmpgui	58

cmpne	59
cmpnei	59
divu	60
eret	60
lb	61
lbu	61
lh	62
lhu	62
lw	63
modu	63
mul	64
muli	64
mv	65
mvhi	65
nor	66
nori	66
not	67
or	67
ori	68
orhi	68
rcsr	69
ret	69
sb	70
scall	70
sextb	71
sexth	71
sh	72
sl	72
sli	73
sr	73
sri	74
sru	74
srui	75
sub	75
sw	76
wcsr	76
xnor	77
xnori	77
xor	78
xori	78
Index	79

LatticeMico32 Processor and Systems

As systems become more complex, there are a growing number of L_2 and L_3 protocols that continue to burden a local host processor. These tend to incrementally add processing requirements to the local processor, starving other critical functions of processor machine cycles. To alleviate the local host processor's processing requirements, embedded processors are being utilized to support the main processor in a distributed processing architecture. These embedded processors offer localized control, OA&M functionality, and statistics gathering and processing features, thereby saving the host processor many unnecessary clock cycles, which can be used for higher-level functions.

A soft processor provides added flexibility in the implementation of your design. Functionality that can be implemented in software rather than hardware allows much greater freedom in terms of the types of changes that can be made. With software-based processing, it is possible for the hardware logic to remain stable and functional upgrades can be made through software modification. Additionally, it is much quicker and simpler to implement functionality in software than it is to design it in hardware, leading to a reduced time to market.

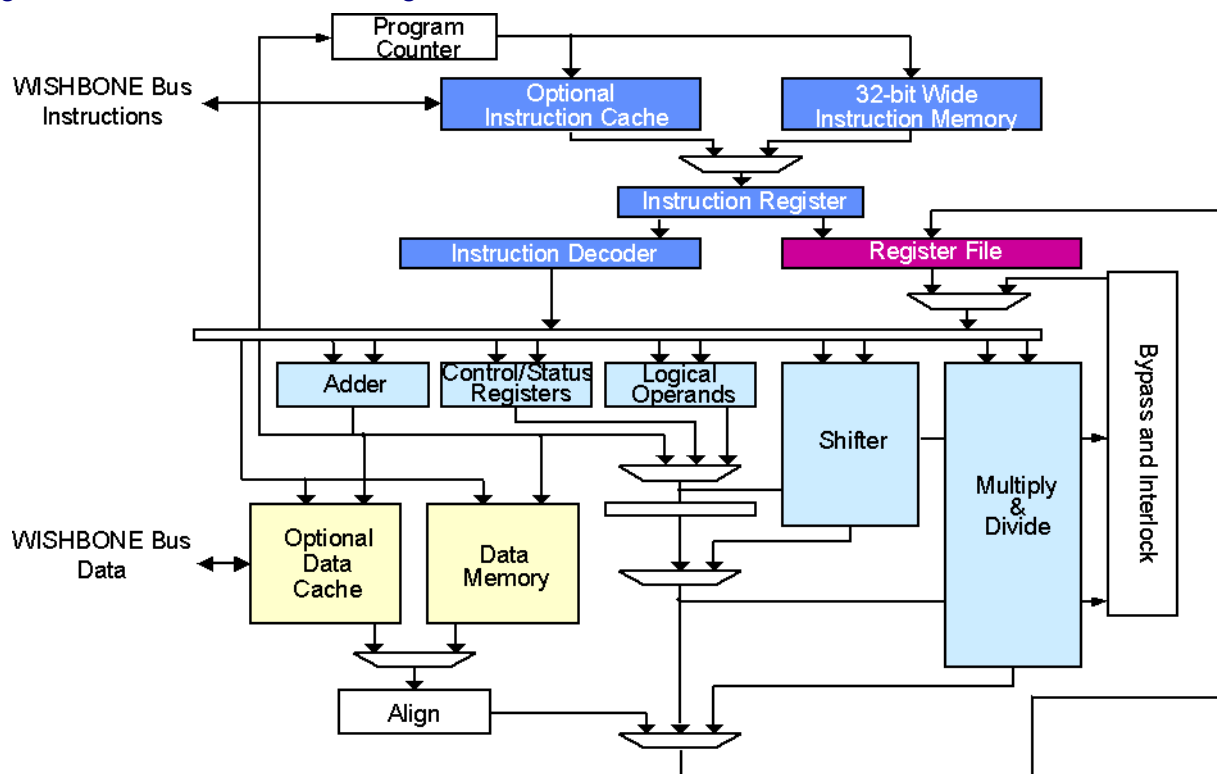
The LatticeMico32™ is a configurable 32-bit soft processor core for Lattice Field Programmable Gate Array (FPGA) devices. By combining a 32-bit wide instruction set with 32 general purpose registers, the LatticeMico32 provides the performance and flexibility suitable for a wide variety of markets, including communications, consumer, computer, medical, industrial, and automotive. With separate instruction and data buses, this Harvard architecture processor allows for single-cycle instruction execution as the instruction and data memories can be accessed simultaneously. Additionally, the LatticeMico32 uses a Reduced Instruction Set Computer (RISC) architecture; thereby, providing a simpler instruction set and faster performance. As a result, the processor core consumes minimal device resources, while maintaining the

performance required for a broad application set. Some of the key features of this 32-bit processor include:

- ◆ RISC architecture
- ◆ 32-bit data path
- ◆ 32-bit instructions
- ◆ 32 general purpose registers
- ◆ Up to 32 external interrupts
- ◆ Optional instruction cache
- ◆ Optional data cache
- ◆ Dual WISHBONE memory interfaces (instruction and data)

Figure 1 shows a block diagram of the LatticeMico32 processor core.

Figure 1: LatticeMico32 Block Diagram



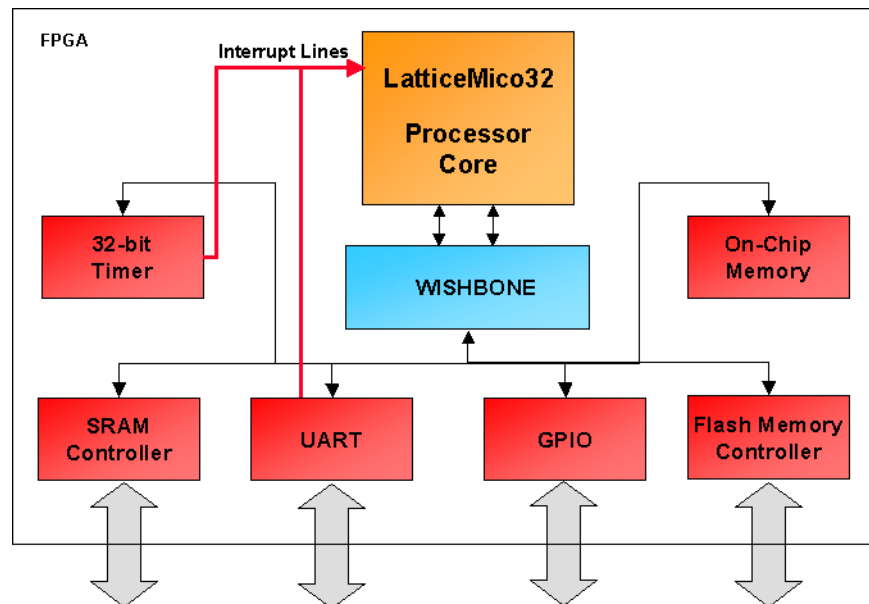
To accelerate the development of processor systems, several optional peripheral components are available with the LatticeMico32 processor. Specifically, these components are connected to the processor via a WISHBONE bus interface, a royalty-free, public domain specification. By using this open source bus interface, you can incorporate your own

WISHBONE components into your embedded designs. The components include:

- ◆ Memory controllers
 - ◆ Asynchronous SRAM
 - ◆ Double Data Rate (DDR)
 - ◆ On-Chip
- ◆ Input/Output (I/O) ports
 - ◆ 32-bit timer
 - ◆ Direct Memory Access (DMA) controller
 - ◆ General Purpose IO (GPIO)
 - ◆ I²C master controller
 - ◆ Serial Peripheral Interface (SPI)
 - ◆ Universal Asynchronous Receiver Transmitter (UART)

Figure 2 shows a complete embedded system using the LatticeMico32 processor along with several components.

Figure 2: LatticeMico32 Processor Embedded System



This manual describes the architecture of the LatticeMico32 processor. It includes information on configuration options, pipeline architecture, register architecture, memory architecture, debug architecture, and the instruction set. It is intended to be used as a reference when you design processors for use on supported Lattice Semiconductor field programmable gate arrays (FPGAs).

Programmer's Model

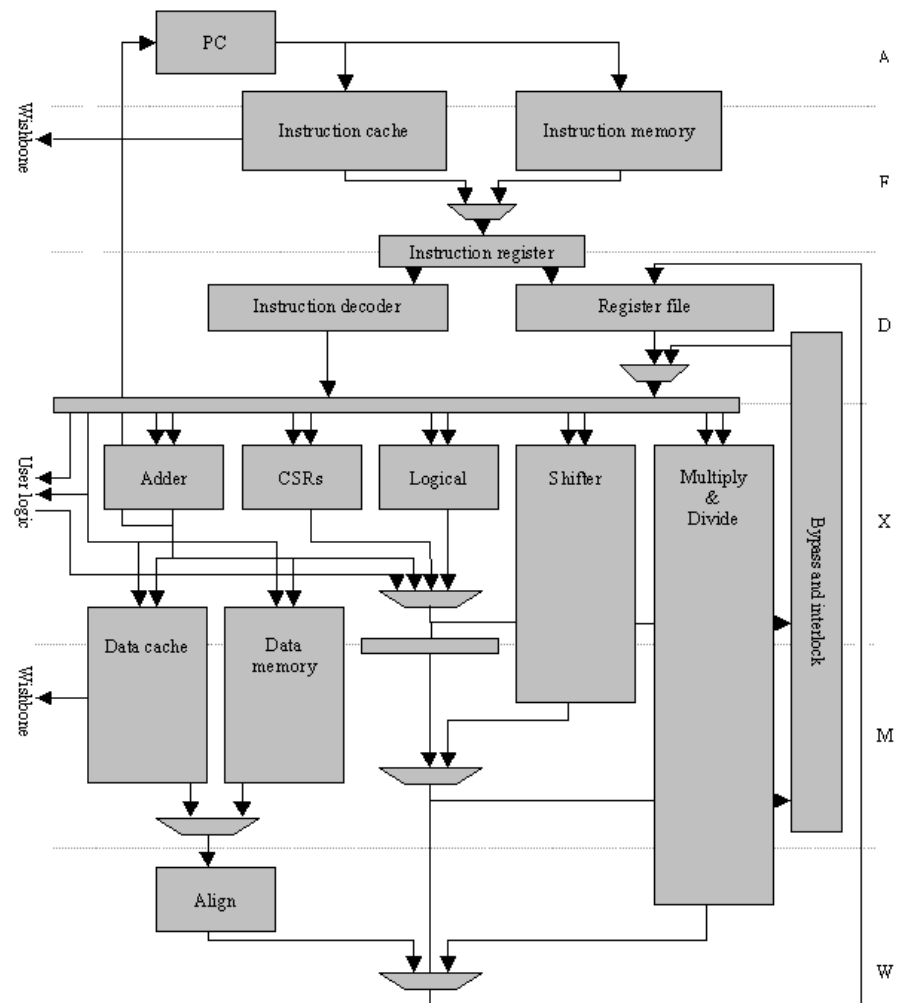
Pipeline Architecture

This section describes the pipeline architecture of the LatticeMico32 processor.

The LatticeMico32 processor uses a 32-bit, 6-stage pipeline, as shown in Figure 3 on page 6. It is fully bypassed and interlocked. The bypass logic is responsible for forwarding results back through the pipeline, allowing most instructions to be effectively executed in a single cycle. The interlock is responsible for detecting read-after-write hazards and stalling the pipeline until the hazard has been resolved. This avoids the need to insert nop directives between dependent instructions, thus keeping code size to a minimum, as well as simplifying assembler-level programming.

The six pipeline stages are:

- ◆ **Address** – The address of the instruction to execute is calculated and sent to the instruction cache.
- ◆ **Fetch** – The instruction is read from memory.
- ◆ **Decode** – The instruction is decoded and operands are either fetched from the register file or bypassed from the pipeline.
- ◆ **Execute** – The operation specified by the instruction is performed. For simple instructions such as addition or a logical operation, execution finishes in this stage, and the result is made available for bypassing.
- ◆ **Memory** – For more complicated instructions such as loads, stores, multiplies, or shifts, a second execution stage is required.
- ◆ **Writeback** – Results produced by the instructions are written back to the register file

Figure 3: LatticeMico32 Pipeline

Data Types

The LatticeMico32 processor supports the data types listed in Table 1.

Table 1: Data Types

Type	Range	Bits	Encoding	C Compiler Type
Unsigned byte	$[0, 2^8-1]$	8	Binary	unsigned char
Signed byte	$[-2^7, 2^7-1]$	8	Two's complement	char
Unsigned half-word	$[0, 2^{16}-1]$	16	Binary	unsigned short
Signed half-word	$[-2^{15}, 2^{15}-1]$	16	Two's complement	short
Unsigned word	$[0, 2^{32}-1]$	32	Binary	unsigned int/ unsigned long
Signed word	$[-2^{31}, 2^{31}-1]$	32	Two's complement	int/long

In addition to the above, the extended data types in Table 2 can be emulated via a compiler.

Table 2: Extended Data Types

Data Type	Range	Bits	Encoding	C Compiler Type
Unsigned double-word	$[0, 2^{64}-1]$	64	Binary	unsigned long long
Signed double-word	$[-2^{63}, 2^{63}-1]$	64	Two's complement	long long
Single-precision real	$[1.1754\text{e-}38, 3.4028\text{e+}38]$	32	IEEE 754	float
Double-precision real	$[2.2250\text{e-}308, 1.7976\text{e+}308]$	64	IEEE 754	double

Register Architecture

This section describes the register architecture of the LatticeMico32 processor.

General Purpose Registers

The LatticeMico32 features the following 32-bit registers:

- ◆ By convention, register 0 (r0) must always hold the value 0, and this is required for correct operation by both the LatticeMico32 assembler and the C compiler. On power-up, value of 0 in r0 is not hardwired and must be initialized by the user to load r0 with the 0 value.
- ◆ Registers 1 through 28 are truly general purpose and can be used as the source or destination register for any instruction. After reset, the values in all of these registers are undefined.
- ◆ Register 29 (ra) is used by the call instruction to save the return address, but is otherwise general purpose.
- ◆ Register 30 (ea) is used to save the value of the Program Counter (PC) when an exception occurs, so it should not be used by user-level programs.
- ◆ Register 31 (ba) saves the value of the Program Counter (PC) when a breakpoint or watchpoint exception occurs, so it should not be used by user-level programs.

After reset, the values in all of the above 32-bit registers are undefined. To ensure that register 0 contains 0, the first instruction executed after reset should be: `xor r0, r0, r0`.

Table 3 below lists the general purpose registers and specifies their use by the C compiler. In this table, the callee is the function called by the caller function.

Table 3: General Purpose Registers

Register Name	Function	Saver
r0	Holds the value zero	
r1	General purpose/Argument 0/Return value 0	Caller
r2	General purpose/Argument 1/Return value 1	Caller
r3	General purpose/Argument 2	Caller
r4	General purpose/Argument 3	Caller
r5	General purpose/Argument 4	Caller
r6	General purpose/Argument 5	Caller
r7	General purpose/Argument 6	Caller
r8	General purpose/Argument 7	Caller
r9	General purpose	Caller
r10	General purpose	Caller
r11	General purpose	Callee
r12	General purpose	Callee
r13	General purpose	Callee
r14	General purpose	Callee
r15	General purpose	Callee
r16	General purpose	Callee
r17	General purpose	Callee
r18	General purpose	Callee
r19	General purpose	Callee
r20	General purpose	Callee
r21	General purpose	Callee
r22	General purpose	Callee
r23	General purpose	Callee
r24	General purpose	Callee
r25	General purpose	Callee
r26/gp	General purpose/Global pointer	Callee
r27/fp	General purpose/Frame pointer	Callee
r28/sp	Stack pointer	Callee
r29/ra	General purpose/Return address	Caller
r30/ea	Exception address	
r31/ba	Breakpoint address	

Control and Status Registers

Table 4 shows all of the Control and Status Register (CSR) names, whether the register can be read or written, and the index used when accessing the register. Some of the registers are optional depending on the configuration of the processor (see “Configuring the LatticeMico32 Processor” on page 31). All signal levels are active high.

Table 4: Control and Status Registers

Name	Access	Index	Optional	Description
PC			No	Program counter
IE	R/W	0x0	Yes	Interrupt enable
IM	R/W	0x1	Yes	Interrupt mask
IP	R	0x2	Yes	Interrupt pending
ICC	W	0x3	Yes	Instruction cache control
DCC	W	0x4	Yes	Data cache control
CC	R	0x5	Yes	Cycle counter
CFG	R	0x6	No	Configuration
EBA	R/W	0x7	No	Exception base address

PC – Program Counter

The PC CSR is a 32-bit register that contains the address of the instruction currently being executed. Because all instructions are four bytes wide, the two least significant bits of the PC are always zero. After reset, the value of the PC CSR is h00000000.

Figure 4: Format of the PC CSR



IE – Interrupt Enable

The IE CSR contains a single-bit flag, IE, that determines whether interrupts are enabled. This flag has priority over the IM CSR. In addition, there are two bits, BIE and EIE, that are used to save the value of the IE field when either a breakpoint or other exception occurs. Each interrupt is associated with a mask bit (IE bit) indexed with each interrupt. After reset, the value of the IE CSR is h00000000.

Figure 5: Format of the IE CSR

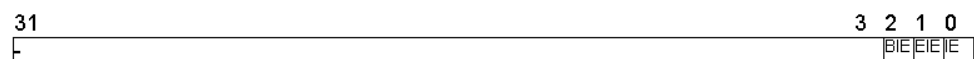


Table 5: Fields of the IE CSR

Field	Values	Description
IE	0 – Interrupts disabled 1 – Interrupts enabled	Determines whether interrupts are enabled.
EIE	0 – Interrupts disabled 1 – Interrupts enabled	Holds a copy of the IE field when an exception occurs.
BIE	0 – Interrupts disabled 1 – Interrupts enabled	Holds a copy of the IE field when a breakpoint occurs.

IM – Interrupt Mask

The IM CSR contains an enable bit for each of the 32 interrupts. Bit 0 corresponds to interrupt 0. In order for an interrupt to be raised, both an enable bit in this register and the IE flag in the IE CSR must be set to 1. After reset, the value of the IM CSR is h00000000.

IP – Interrupt Pending

The IP CSR contains a pending bit for each of the 32 interrupts. A pending bit is set when the corresponding interrupt request line is asserted low. Bit 0 corresponds to interrupt 0. Bits in the IP CSR can be cleared by writing a 1 with the wcsr instruction. Writing a 0 has no effect. After reset, the value of the IP CSR is h00000000.

ICC – Instruction Cache Control

The ICC CSR provides a control bit that, when written with any value, causes the contents of the entire instruction cache to be invalidated.

Figure 6: Format of the ICC CSR

Field	Values	Description
I	Any – Invalidate instruction cache	When written, the contents of the instruction cache are invalidated

DCC – Data Cache Control

The DCC CSR provides a control bit that, when written with any value, causes the contents of the entire data cache to be invalidated.

Figure 7: Format of the DCC CSR



Table 6: Fields of the DCC CSR

Field	Values	Description
I	Any – Invalidate data cache	When written, the contents of the data cache are invalidated

CC – Cycle Counter

The CC CSR is an optional 32-bit register that is incremented on each clock cycle. This can be used to profile ghost code sequences.

CFG – Configuration

The CFG CSR details the configuration of a particular instance of a LatticeMico32 processor.

Figure 8: Format of the CFG CSR

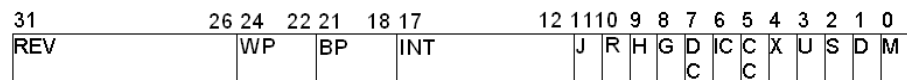


Table 7: Fields of the CFG CSR

Field	Values	Description
M	0 – Multiply is not implemented 1 – Multiply is implemented	Indicates whether a hardware multiplier is implemented.
D	0 – Divide is not implemented 1 – Divide is implemented	Indicates whether a hardware divider is implemented.
S	0 – Barrel shift is not implemented 1 – Barrel shift is implemented	Indicates whether a hardware barrel-shifter is implemented.
U		Reserved.
X	0 – Sign extend is not implemented 1 – Sign extend is implemented	Indicates whether the sign-extension instructions are implemented.
CC	0 – Cycle counter is not implemented 1 – Cycle counter is implemented	Indicates whether the CC CSR is implemented.
IC	0 – Instruction cache is not implemented 1 – Instruction cache is implemented	Indicates whether an instruction cache is implemented.

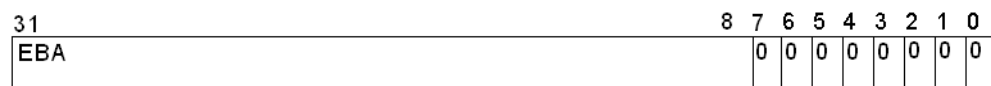
Table 7: Fields of the CFG CSR (Continued)

Field	Values	Description
DC	0 – Data cache is not implemented 1 – Data cache is implemented	Indicates whether a data cache is implemented.
G	0 – Debug is not implemented 1 – Data cache is implemented	Indicates whether software-based debug support is implemented.
H	0 – H/W debug is not implemented 1 – H/W debug is implemented	Indicates whether hardware-based debug support is implemented.
R	0 – ROM debug is not implemented 1 – ROM debug is implemented	Indicates whether support for debugging ROM-based programs is implemented.
J	0 – JTAG UART is not implemented 1 – JTAG UART is implemented	Indicates whether a JTAG UART is implemented.
INT	0 – 32	Indicates the number of external interrupts.
BP	0 – 4	Indicates the number of breakpoint CSRs.
WP	0 – 4	Indicates the number of watchpoint CSRs.
REV	0 – 63	Processor Revision number. This is set automatically. You cannot reset this field.

EBA – Exception Base Address

The EBA CSR specifies the base address of the exception handlers. After reset, the value of EBA is set to EBA_RESET. If you write a value to the register where the lower byte is not zero, it will read back all zeros. So, there is no need for you to mask zeros to avoid issues.

Figure 9: Format of the EBA CSR



Memory Architecture

This section describes the memory architecture of the LatticeMico32 processor.

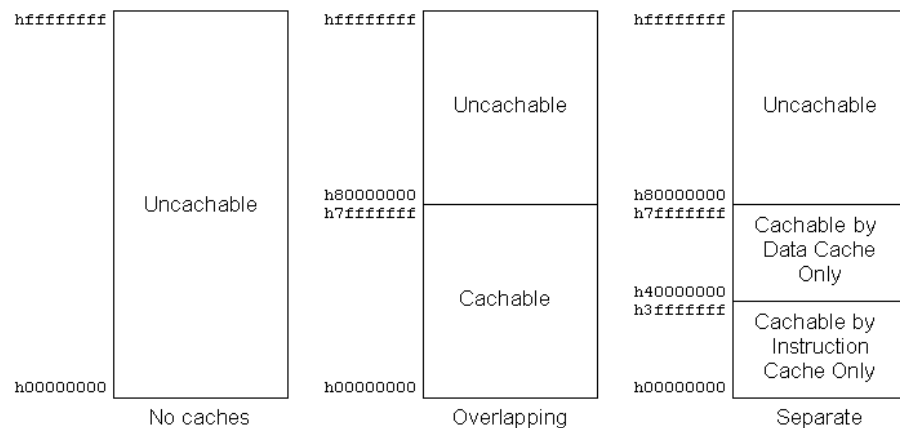
Address Space

The LatticeMico32 processor has a flat 32-bit, byte-addressable address space. For LatticeMico32 processors with caches, the portion of the address space that is cacheable can be configured separately for both the instruction and data cache. This allows for the size of the cache tag RAMs to be optimized to be as small as is required (the fewer the number of cacheable addresses, the smaller the tag RAMs will be).

If an instruction cache is used, attempts to fetch instructions from outside of the range of cacheable addresses results in undefined behavior. So, only one cached region is supported. Portions of the memory image are not cached, so if a miss occurs, it will not be fetched.

Figure 10 illustrates some possible configurations. Typically, the parts of the address space that are cacheable are used for storing code or program data, with I/O components being mapped into uncachable addresses.

Figure 10: Cacheable Addresses



Endianness

The LatticeMico32 processor is big-endian, which means that multi-byte objects, such as half-words and words, are stored with the most significant byte at the lowest address.

Address Alignment

All memory accesses must be aligned to the size of the access, as shown in Table 8. There is no check performed for unaligned access. All unaligned accesses result in undefined behaviour.

Table 8: Memory Access Alignment Requirements

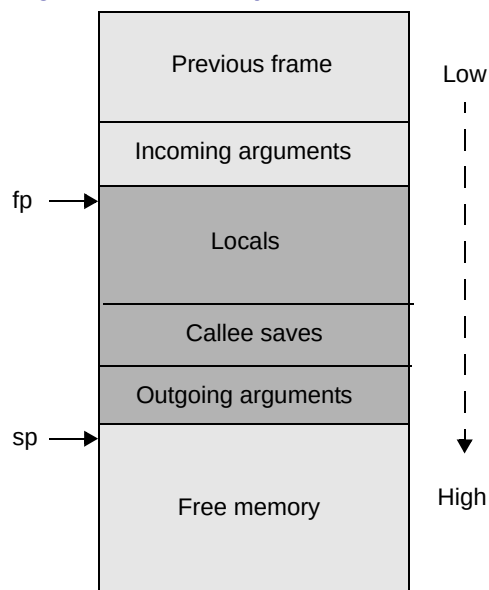
Access Size	Address Requirements
Byte	None
Half-word	Address must be half-word aligned (bit 0 must be 0)
Word	Address must be word aligned (bits 1 and 0 must be 0)

Stack Layout

Figure 11 shows the conventional layout of a stack frame. The stack grows toward higher memory as it moves downwards. The stack pointer (sp) points to the first unused location, and the frame pointer (fp) points at the first location used in the active frame. In many cases, a compiler may be able to eliminate the frame pointer, because data can often be accessed by using a negative displacement from the stack pointer, freeing up the frame pointer for use as a general purpose register.

As illustrated in Table 3 on page 8, the first eight function arguments are passed in registers. Any remaining arguments are passed on the stack as illustrated in Figure 11.

Figure 11: Stack Layout



Caches

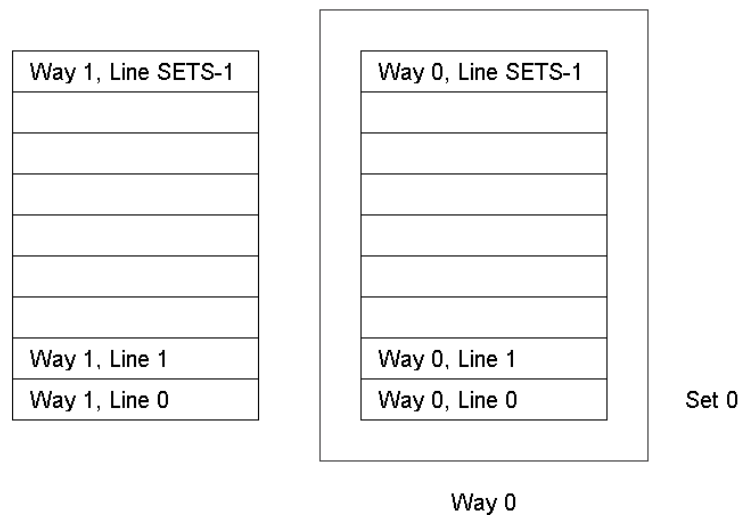
A cache is a fast memory (single-cycle access) that stores a copy of a limited subset of the data held in main memory, which may take the CPU several cycles to access. A cache helps improve overall performance by exploiting the fact that the same data is typically accessed several times in a short interval. By storing a local copy of the data in the processor's cache, the multiple cycles required to access the data can be reduced to just a single cycle for all subsequent accesses once the data is loaded into the cache.

Cache Architecture

A cache can also exploit the fact that when accessing a data item, data at adjacent addresses are also likely to be accessed to (such as with arrays or structures). It achieves this by loading data into the cache in lines. A line can consist of 4, 8, or 16 adjacent bytes, and is specified by the `BYTES_PER_LINE` option.

A one-way associative (direct-mapped) cache consists of an array of cache lines known as a "way." To allow the cache to operate at a high frequency, data from main memory can only be stored in a specific cache line. A two-way associative cache consists of a two-dimensional array of cache lines. This requires slightly more logic to implement, but allows data from main memory to be stored in one of two places in the cache. This helps performance by reducing cache conflicts that occur when a program is accessing multiple data items that would map to the same cache line in a one-way associative cache. The number of lines in each way is specified by the `ICACHE_SETS` and `DCACHE_SETS` options. The ways are assigned in a round-robin fashion. Each time a cache miss occurs the way number is switched.

Figure 12: Cache Organization



The LatticeMico32 caches are write-through. This means that whenever a store instruction writes to an address that is cached, the data is written to both the cache and main memory. A read-miss allocation policy means that a cache line is only fetched from memory for a load instruction. If a cache miss

occurs for a store instruction, the data is written directly to memory without the cache being updated.

The LatticeMico32 processor supports a range of cache configurations, as detailed in Table 9.

Table 9: Cache Configurations

Attribute	Values
Size	0 kB, 1 kB, 2 kB, 4 kB, 8 kB, 16 kB, 32 kB
Sets	128, 256, 512, 1024
Associativity	1, 2
Bytes-per-line	4, 8, 16
Write policy	Write-through
Update policy	Read miss only

The LatticeMico32 caches are initialized automatically by embedded logic and thus do not require a program to initialize or enable them.

Invalidating the Caches

The contents of the instruction cache can be invalidated by writing to the ICC CSR. It is recommended to follow the write to the ICC CSR with four nops, as follows:

```
wcsr ICC, r0
nop
nop
nop
nop
```

The contents of the data cache can similarly be invalidated by writing to the DCC CSR, as follows:

```
wcsr DCC, r0
```

It is recommended to avoid placing a load or store instruction immediately before or after the wcsr instruction.

The LatticeMico32 caches are not kept consistent with respect to each other. This means that if a store instruction writes to an area of memory that is currently cached by the instruction cache, then the instruction cache will not be automatically updated to reflect the store. It is the responsibility of the software programmer to invalidate the instruction cache after the write has taken place, if necessary.

Similarly, the caches do not snoop bus activity to monitor for writes by peripherals (by DMA for example) to addresses that are cached. It is again the responsibility of the software programmer to insure that the cache is invalidated before reading memory that may have been written by a peripheral.

Exceptions

Exceptions are events either inside or outside of the processor that cause a change in the normal flow of program execution. The LatticeMico32 processor can raise eight types of exceptions as shown in Table 10. Note that one clock must be held in active high until an acknowledge (ACK_I) is accepted. The remaining are internal and no ACK_I is generated. The exceptions are listed in a decreasing order of priority, so if multiple exceptions occur simultaneously, the exception with the highest priority is raised.

Table 10: Exceptions

Exception	ID	Condition
Reset	0	Raised when the processor's reset pin is asserted.
Breakpoint	1	Raised when either a break instruction is executed or when a hardware breakpoint is triggered.
InstructionBusError	2	Raised when an instruction fetch fails, typically due to the requested address being invalid.
Watchpoint	3	Raised when a hardware watchpoint is triggered
DataBusError	4	Raised when a data access fails, typically due to either the requested address being invalid or the type of access not being allowed.
DivideByZero	5	Raised when an attempt is made to divide by zero.
Interrupt	6	Raised when one of the processor's interrupt pins is asserted, providing that the corresponding field in the interrupt mask (IM) CSR is set and the global interrupt enable flag, IE.IE, is set.
SystemCall	7	Raised when a scall instruction is executed.

Exception Processing

Exceptions occur in the Execute pipeline stage. If there is an instruction in the Memory pipeline stage, that instruction is first allowed to finish. All instructions from the Execute stage back are then killed and do not cause any user-transparent state changes. For example, no flags are set.

DataBusError exceptions are imprecise, so the exception may not necessarily be raised on the load or store instruction that caused the bus error (it will be raised on some later instruction). This allows for high-performance implementations where the pipeline need not be stalled until a store instruction commits to memory. This imprecise behavior is acceptable because bus errors are non-recoverable.

The sequence of operations performed by the processor after an exception depends upon the type of the highest priority exception that has occurred.

Non-Debug Exceptions

When a non-debug related exception occurs, the address of the instruction in the Execute stage of the processor's pipeline is saved in the ea register. The global interrupt enable held in the IE.IE register is copied to the IE.EIE register and then cleared, disabling interrupts. Finally, the PC is set to the address of the exception handler, which is derived from the address contained in the EBA, if the DC.RE bit is not set, or the DEBA, if the DC.RE bit is set.

```
ea = PC
IE.EIE = IE.IE
IE.IE = 0
PC = (DC.RE ? DEBA:EBA) + (ID * 32)
```

Debug Exceptions

When a debug-related exception such as a Breakpoint or Watchpoint exception occurs, the address of the instruction in the Execute stage of the processor's pipeline is saved in the ba register. The global interrupt enable held in the IE.IE register is copied to the IE.BIE register and then cleared, disabling interrupts. Finally, the PC is set to the address of the exception handler, which is derived from the address contained in the DEBA.

```
ba = PC
IE.BIE = IE.IE
IE.IE = 0
PC = DEBA + (ID * 32)
```

Nesting Exceptions

Because different registers are used to save a state when a debug-related exception occurs (ba and IE.BIE instead of ea and IE.EIE), limited nesting of exceptions is possible. This allows the interrupt handler code to be debugged. Any further nesting of exceptions requires software support.

Interrupts

The LatticeMico32 processor supports up to 32 maskable, active-low, level-sensitive interrupts. Each interrupt line has a corresponding mask bit in the IM CSR. The Mask Enable is active high. A global interrupt enable flag is implemented in the IE CSR. Software can query the status of the interrupts and acknowledge them through the IP CSR.

If more interrupt sources or more sophisticated interrupt detection methods are required, external interrupt controllers can be cascaded onto the processors interrupt pins to provide the needed functionality.

Exception Handlers

When an exception occurs, the CPU branches to an address that is an offset from either the EBA CSR or the DEBA CSR. The offset is calculated by multiplying the exception ID by 32. Exception IDs are shown in Figure 10 on page 17. As all LatticeMico32 instructions are 4 bytes long, this means each exception handler can be eight instructions long. If further instructions are required, the handler can call a subroutine.

Whether the EBA or DEBA is used as the base address depends upon the type of the exception that occurred, and whether DC.RE is set or not. Having

two different base addresses for the exception table allows a debug monitor to exist in a different memory from the main program code. For example, the debug monitor may exist in an on-chip ROM, whereas the main program code may be in a DDR or SRAM. The DC.RE flag, allows either interrupts to run at full speed when debugging or for the debugger to take complete control and handle all exceptions.

When an exception occurs, the only state that is automatically saved by the CPU is the PC, which is saved in either ea or ba, and the interrupt enable flag, IE.IE, which is saved in either IE.EIE or IE.BIE. It is the responsibility of the exception handler to save and restore any other registers that it uses, if it returns to the previously executing code. This will likely be the case for interrupt exception handlers, but may not be so for handlers that typically result in the termination of a program, such as divide by zero or bus error handlers.

The following code snippet shows how the exception handlers can be implemented. The nops are required to insure that the next exception handler is aligned at the correct address.

Figure 13: Exception Handler Example

```
/* Exception handlers */

_reset_handler:
    xor    r0, r0, r0
    bi     _crt0
    nop
    nop
    nop
    nop
    nop
    nop
    nop

_breakpoint_handler:
    sw     (sp+0), ra
    calli  save_all
    mvi    r1, SIGTRAP
    calli  raise
    bi     restore_all_and_bret
    nop
    nop
    nop

_instruction_bus_error_handler:
    sw     (sp+0), ra
    calli  save_all
    mvi    r1, SIGSEGV
    calli  raise
    bi     restore_all_and_eret
    nop
    nop
    nop
```

Figure 13: Exception Handler Example (Continued)

```
_watchpoint_handler:
    sw      (sp+0), ra
    calli   save_all
    mvi     r1, SIGTRAP
    calli   raise
    bi      restore_all_and_bret
    nop
    nop
    nop

_data_bus_error_handler:
    sw      (sp+0), ra
    calli   save_all
    mvi     r1, SIGSEGV
    calli   raise
    bi      restore_all_and_eret
    nop
    nop
    nop

_divide_by_zero_handler:
    sw      (sp+0), ra
    calli   save_all
    mvi     r1, SIGFPE
    calli   raise
    bi      restore_all_and_eret
    nop
    nop
    nop

_interrupt_handler:
    sw      (sp+0), ra
    calli   save_all
    mvi     r1, SIGINT
    calli   raise
    bi      restore_all_and_eret
    nop
    nop
    nop

_system_call_handler:
    sw      (sp+0), ra
    calli   save_all
    mv      r1, sp
    calli   handle_scall
    bi      restore_all_and_eret
    nop
    nop
    nop
```

Figure 13: Exception Handler Example (Continued)

```

_save_all:
    addi    sp, sp, -56
    /* Save all caller save registers onto the stack */
    sw      (sp+4), r1
    sw      (sp+8), r2
    sw      (sp+12), r3
    sw      (sp+16), r4
    sw      (sp+20), r5
    sw      (sp+24), r6
    sw      (sp+28), r7
    sw      (sp+32), r8
    sw      (sp+36), r9
    sw      (sp+40), r10
    sw      (sp+48), ea
    sw      (sp+52), ba
    /* ra needs to be moved from initial stack location */
    lw      r1, (sp+56)
    sw      (sp+44), r1
    ret

/* Restore all registers and return from exception */
_restore_all_and_eret:
    lw      r1, (sp+4)
    lw      r2, (sp+8)
    lw      r3, (sp+12)
    lw      r4, (sp+16)
    lw      r5, (sp+20)
    lw      r6, (sp+24)
    lw      r7, (sp+28)
    lw      r8, (sp+32)
    lw      r9, (sp+36)
    lw      r10, (sp+40)
    lw      ra, (sp+44)
    lw      ea, (sp+48)
    lw      ba, (sp+52)
    addi    sp, sp, 56
    eret

/* Restore all registers and return from breakpoint */
_restore_all_and_bret:
    lw      r1, (sp+4)
    lw      r2, (sp+8)
    lw      r3, (sp+12)
    lw      r4, (sp+16)
    lw      r5, (sp+20)
    lw      r6, (sp+24)
    lw      r7, (sp+28)
    lw      r8, (sp+32)
    lw      r9, (sp+36)
    lw      r10, (sp+40)
    lw      ra, (sp+44)
    lw      ea, (sp+48)
    lw      ba, (sp+52)
    addi    sp, sp, 56
    bret

```

To ensure that this code is at the correct address, it is common practice to place it in its own section. This can be achieved with the following assembler directive, which should be placed at the start of the code:

```
.section    .boot, "ax", @progbits
```

Then in the linker script, the code can be placed at the reset value of EBA or DEBA as shown in Figure 14.

Figure 14: Placing Exception Handler in Memory

```
MEMORY
{
    ram : ORIGIN = 0x00000000, LENGTH = 0x00100000
}

SECTIONS
{
    .boot : { *(.boot) } > ram
}
```

Nested Exceptions

If desirable, with a small amount of software code, it is possible to support nested exceptions. There are two situations where this may be needed:

- ◆ To allow system calls to be callable from exception handlers
- ◆ To prioritize interrupts

To enable nested exceptions, an exception handler must save all the state that is modified when an exception occurs. This includes the ea and ba registers, as well as the IE CSR. These registers can simply be saved on the stack. When returning from the exception handler, these registers must, obviously, be restored from the values saved on the stack.

To support nested prioritized interrupts, after saving the registers outlined above, the IM CSR should also be saved, and then all lower priority interrupts masked. IE.IE can then be set to re-enable interrupts. When the interrupt handler has finished, IE.IE should be cleared, before all the saved registers, including IM, are restored.

Remapping the Exception Table

In order to increase performance, the exception table can be remapped at runtime, by writing a new value to EBA. This would be used in a system where the power-up value of EBA points to a slow, non-volatile memory, such as a FLASH memory, but code is executed from a faster, non-volatile RAM, such as DDR or SRAM.

Reset Summary

During reset, the following occurs:

- ◆ All CSRs are set to their reset values as listed in “Control and Status Registers” on page 9.
- ◆ Interrupts are disabled.
- ◆ All hardware breakpoints and watchpoints are disabled.
- ◆ If implemented, the contents of the caches are invalidated.
- ◆ A reset exception is raised, which causes the PC to be set to the value in the EBA CSR, where program execution starts.

Note that the register file is not reset, so it is the responsibility of the reset exception handler to set register 0 to 0. This should be achieved by executing:
`xor r0, r0, r0.`

Using Breakpoints

The LatticeMico32 architecture supports both software and hardware breakpoints. Software breakpoints should be used for setting breakpoints in code that resides in volatile memory, such as DDR or SRAM, while hardware breakpoints should be used for setting breakpoints in code that resides in non-volatile memory such as FLASH or ROM.

A software breakpoint is simply a break instruction. In order to set a breakpoint, it is simply a case of replacing the instruction at the desired address with the break instruction. When the break instruction is executed, a breakpoint exception will be raised, and the ba register will contain the address of the break instruction that was executed. It will then be up to the exception handler to either restore the instruction that was overwritten and continue execution, or to take some other action, depending upon why the breakpoint was set.

It is typically either not possible or very slow to write a break instruction to non-volatile RAM. For processors with `BREAKPOINTS > 0`, it is possible to set a hardware breakpoint by writing the address of the instruction on which the breakpoint should be set, to one of the `BPn` CSRs. The processor then constantly compares the values in these `BPn` CSRs with the address of the instruction being executed. If a match occurs, and the breakpoint is enabled (by the LSB being set to 1), then a breakpoint exception will be raised. As with software breakpoints, the address of the instruction that caused the breakpoint will be saved in the ba register. If the breakpoint exception handler wishes to resume program execution, it must clear the enable bit in the relevant BP CSR, otherwise the breakpoint exception will be raised as soon as execution resumes.

Using Watchpoints

The LatticeMico32 architecture supports hardware watchpoints. Watchpoints are a mechanism by which a program can watch out for specific memory accesses. For example, a program can set up a watchpoint that will cause a watchpoint exception to be raised every time the address 0 is accessed (something that is useful for tracking down NULL pointer errors in C programs).

To set up a watchpoint, the memory address that is being watched must be written to one of the WP n CSRs. The watchpoint then needs to be enabled by writing the corresponding C field in the DC CSR. This field takes one of four values that indicate: the watchpoint is disabled, the watchpoint exception will only be raised on read accesses, the watchpoint exception will only be raised on write accesses, or finally, the watchpoint exception will be raised on either read or write accesses.

Debug Architecture

This section describes the debug architecture of the LatticeMico32 processor.

The LatticeMico32 debug architecture provides:

- ◆ Software breakpoints
- ◆ Hardware breakpoints
- ◆ Hardware watchpoints
- ◆ Single-step capability
- ◆ Ability to remap exception handlers when debugging is enabled
- ◆ Hardware support for debugging interrupt handlers

Table 11 shows the Debug Control And Status Registers.

Table 11: Debug Control and Status Registers

Name	Access	Index	Description
DC	W	0x8	Debug control
DEBA	R/W	0x9	Debug exception base address
JTX	R/W	0xe	JTAG UART transmit
JRX	R/W	0xf	JTAG UART receive
BP0	W	0x10	Breakpoint address 0
BP1	W	0x11	Breakpoint address 1
BP2	W	0x12	Breakpoint address 2
BP3	W	0x13	Breakpoint address 3
WP0	W	0x18	Watchpoint address 0
WP1	W	0x19	Watchpoint address 1

Table 11: Debug Control and Status Registers (Continued)

Name	Access	Index	Description
WP2	W	0x1a	Watchpoint address 2
WP3	W	0x1b	Watchpoint address 3

DC – Debug Control

The DC CSR contains flags that control debugging facilities. After reset, the value of the DC CSR is h00000000. This CSR is only implemented if `DEBUG_ENABLED` equals `TRUE`.

Figure 15: Format of the DC CSR**Table 12: Fields of the DC CSR**

Field	Value	Description
SS	0 – Single step disabled 1 – Single step enabled	Determines whether single-stepping is enabled
RE	0 – Remap only debug exceptions 1 – Remap all exceptions	Determines whether all exceptions are remapped to the base address specified by DEBA or just debug exceptions
Cn	b00 – Watchpoint <i>n</i> disabled b01 – Break on read b10 – Break on write b11 – Break on read or write	Enable for corresponding Wpn CSR

DEBA – Debug Exception Base Address

The DEBA CSR specifies the base address of the debug exception handlers. After reset, the value of the DEBA CSR is set to `DEBA_RESET`. This CSR is only implemented if `DEBUG_ENABLED` equals `TRUE`.

Figure 16: Format of the DEBA CSR

JTX – JTAG UART Transmit Register

The JTX CSR can be used for transmitting data through a JTAG interface. This CSR is only implemented if JTAG_UART_ENABLED equals TRUE.

Figure 17: Format of the JTX CSR



Table 13: Fields of the JTX CSR

Field	Values	Description
TXD		Transmits data
F	0 – Empty 1 – Full	Indicates whether the transmit data register is full

JRX – JTAG UART Receive Register

The JRX CSR can be used for receiving data through a JTAG interface. This CSR is only implemented if JTAG_UART_ENABLED equals TRUE.

Figure 18: Format of the JRX CSR

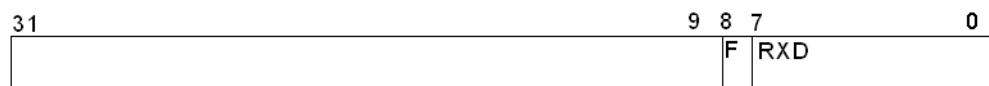


Table 14: Fields of the JTX CSR

Field	Values	Description
RXD		Receives data
F	0 – Empty 1 – Full	Indicates whether the receive data register is full

BPn – Breakpoint

The BPn CSRs hold an instruction breakpoint address and a control bit that determines whether the breakpoint is enabled. Because instructions are always word-aligned, only the 30 most significant bits of the breakpoint address are needed. After reset, the value of the BPn CSRs is h00000000.

These CSRs are only implemented if DEBUG_ENABLED equals TRUE.

Figure 19: Format of the BPn CSRs



Table 15: BPn CSR Fields

Field	Value	Description
E	b0 – Breakpoint is disabled	Breakpoint enable
	b1 – Breakpoint is enabled	
A		Breakpoint address (Bits 31:2)

WPn – Watchpoint

The WPn CSRs hold data watchpoint addresses. After reset, the value of the WPn CSRs is h00000000. These CSRs are only implemented if DEBUG_ENABLED equals TRUE.

Instruction Set Categories

LatticeMico32 supports a variety of instructions for arithmetic, logic, data comparison, data movement, and program control. Not all instructions are available in all configurations of the processor. Support for some types of instructions can be eliminated to reduce the amount of FPGA resources used. See “Configuring the LatticeMico32 Processor” on page 31.

Instructions ending with the letter *i* use an immediate value instead of a register. Instructions ending with *hi* use a 16-bit immediate and the high 16 bits from a register. Instructions ending with the letter *u* treat the data as unsigned integers.

For descriptions of individual instructions, see “Instruction Set” on page 43.

Arithmetic

The instruction set includes the standard 32-bit integer arithmetic operations. Support for the multiply and divide instructions is optional.

- ◆ Add: add, addi
- ◆ Subtract: sub
- ◆ Multiply: mul, muli
- ◆ Divide and modulus: divu, modu

There are also instructions to sign-extend byte and half-word data to word size. Support for these instructions is optional.

- ◆ Sign-extend: sextb, sixth

Logic

The instruction set includes the standard 32-bit bitwise logic operations. Most of the logic instructions also have 16-bit immediate or high 16-bit versions.

- ◆ AND: and, andi, andhi
- ◆ OR: or, ori, orhi

- ◆ Exclusive-OR: xor, xori
- ◆ Complement: not
- ◆ NOR: nor, nori
- ◆ Exclusive-NOR: xnor, xnori

Comparison

The instruction set has basic comparison instructions with versions for register-to-register and register-to-16-bit-immediate and signed and unsigned comparisons. The instructions return 1 if true and 0 if false.

- ◆ Equal: cmpe, cmpei
- ◆ Not equal: cmpne, cmpnei
- ◆ Greater: cmpg, cmpgi, cmpgu, cmpgui
- ◆ Greater or equal: cmpge, cmpgei, cmpgeu, cmpgeui

Shift

The instruction set supports left and right shifting of data in general purpose registers. The number of bits to shift can be given through a register or a 5-bit immediate. The right shift instruction has signed and unsigned versions (also known as arithmetic and logical shifting). Support for shift instructions is optional.

- ◆ Left shift: sl, sli
- ◆ Right shift: sr, sri, sru, srui

Data Transfer

Data transfer includes instructions that move data of byte, half-word, and word sizes between memory and registers. Memory addresses are relative and given as the sum of a general purpose register and a signed 16-bit immediate. For example: (r2+32).

- ◆ Load register from memory: lb, lbu, lh, lhu, lw
Byte and half-word values are either sign-extended or zero-extended to fill the register.
- ◆ Store register to memory: sb, sh, sw
Byte and half-word values are taken from the lowest order part of the register.

There are also instructions for moving data from one register to another, including general purpose and control and status registers.

- ◆ Move between general purpose registers: mv
- ◆ Move immediate to high 16 bits of register: mvhi
- ◆ Read and write control and status register: rcsr, wcsr

Program Flow Control

Program flow control instructions include branches, function and exception calls, and returns. The conditional branches and the immediate versions of the unconditional branch and call instructions establish the next instruction's address by adding a signed immediate to the PC register. Since the immediate is signed, the jump can be to a lower or higher address.

- ◆ Unconditional branch: b, bi
- ◆ Branch if equal: be
- ◆ Branch if not equal: bne
- ◆ Branch if greater: bg, bgu
- ◆ Branch if greater or equal: bge, bgeu
- ◆ Function call and return: call, calli, ret
- ◆ System call: scall
- ◆ Return from exception: eret
- ◆ Software breakpoint and return: break, bret

Configuring the LatticeMico32 Processor

This chapter describes possible configuration options that you can use for the LatticeMico32 processor. We expect you to use the Lattice Mico System Builder (MSB) tool to configure the LatticeMico32 processor. Use the processor's configuration GUI, located in the MSB, to specify the Verilog parameters of the processor's RTL. For more information on the processor's Configuration GUI, please refer to our online Help.

Configuration Options

Table 16 describes the Verilog parameters for the LatticeMico32 processor.

Table 16: Verilog Configuration Options

Parameter Name	Values	Default	Description
MC_MULTIPLY_ENABLED	TRUE, FALSE	FALSE	Enables LUT-based multicycle multiplier. mul, muli instructions are implemented. Multiply instructions take 32 cycles to complete.
PL_MULTIPLY_ENABLED	TRUE, FALSE	TRUE	Enables pipelined multiplier (uses DSP blocks if available). mul, muli instructions are implemented. Multiply instructions take 3 cycles to complete.
DIVIDE_ENABLED	TRUE, FALSE	FALSE	Determines whether the divide and modulus instructions (divu, modu) are implemented.

Table 16: Verilog Configuration Options (Continued)

Parameter Name	Values	Default	Description
MC_BARREL_SHIFT_ENABLED	TRUE, FALSE	FALSE	Enables LUT-based multicycle barrel shifter. Enables shift instructions (sr, sri, sru, srui, sl, sli). Each shift instruction can take up to 32 cycles. If both SIGN_EXTEND_ENABLED and PL_BARREL_SHIFT_ENABLED are FALSE, this option must be set to TRUE.
PL_BARREL_SHIFT_ENABLED	TRUE, FALSE	TRUE	Enables pipelined barrel shifter. Enables shift instructions (sr, sri, sru, srui, sl, sli). Shift instructions take 3 cycles to complete. If both MC_BARREL_SHIFT_ENABLED and SIGN_EXTEND_ENABLED are FALSE, this option must be set to TRUE.
SIGN_EXTEND_ENABLED	TRUE, FALSE	FALSE	Determines whether the sign-extension instructions (sextb, sexth) are implemented. If both MC_BARREL_SHIFT_ENABLED and PL_BARREL_SHIFT_ENABLED are FALSE, this option must be set to TRUE.
DEBUG_ENABLED	TRUE, FALSE	TRUE	Determines whether software-based debugging support is implemented (that is, a ROM monitor is required to debug).
HW_DEBUG_ENABLED	TRUE, FALSE	TRUE	Determines whether hardware-based debugging support is implemented (that is, a ROM monitor is not required to debug). If this option is set to TRUE, DEBUG_ENABLED and JTAG_ENABLED must also be set to TRUE.
ROM_DEBUG_ENABLED	TRUE, FALSE	FALSE	Determines whether support for debugging ROM-based programs is implemented. If this option is set to TRUE, DEBUG_ENABLED must also be set to TRUE.
BREAKPOINTS	0-4		Specifies the number of breakpoint CSRs. If this option is set to a non-zero value, ROM_DEBUG_ENABLED must be set to TRUE.
WATCHPOINTS	0-4		Specifies the number of watchpoint CSRs. If this option is set to a non-zero value, SW_DEBUG_ENABLED must be set to TRUE.
JTAG_ENABLED	TRUE, FALSE	TRUE	Determines whether a JTAG interface is implemented.
JTAG_UART_ENABLED	TRUE, FALSE	TRUE	Determines whether a JTAG UART is implemented. If this option is set to TRUE, JTAG_ENABLED must be set to TRUE.
CYCLE_COUNTER_ENABLED	TRUE, FALSE	FALSE	Determines whether a cycle counter is implemented.
ICACHE_ENABLED	TRUE, FALSE	TRUE	Determines whether an instruction cache is implemented.

Table 16: Verilog Configuration Options (Continued)

Parameter Name	Values	Default	Description
ICACHE_BASE_ADDRESS	Any address aligned to the size of the cacheable region.	0	Specifies the base address of region cacheable by instruction cache.
ICACHE_LIMIT	Any integer multiple of the capacity of the cache added to the base address of the cacheable region	0x7FFFFFFF	Specifies the upper limit of region cacheable by instruction cache.
ICACHE_SETS	128, 256, 512, 1024	512	Specifies the number of sets in the instruction cache.
ICACHE_ASSOCIATIVITY	1, 2	1	Specifies the associativity of instruction cache.
ICACHE_BYTES_PER_LINE	4, 8, 16	4	Specifies the number of bytes per instruction cache line.
DCACHE_ENABLED	TRUE, FALSE	TRUE	Determines whether a data cache is implemented.
DCACHE_BASE_ADDRESS	Any address aligned to the size of the cacheable region	0	Specifies the base address of region cacheable by data cache.
DCACHE_LIMIT	Any integer multiple of the capacity of the cache added to the base address of the cacheable region	0xFFFFFFFF	Specifies the upper limit of region cacheable by data cache.
DCACHE_SETS	128, 256, 512, 1024	512	Specifies the number of sets in the data cache.
DCACHE_ASSOCIATIVITY	1, 2	1	Specifies the associativity of data cache.
DCACHE_BYTES_PER_LINE	4, 8, 16	4	Specifies the number of bytes per data cache line.
INTERRUPTS	0–32	32	Specifies the number of external interrupts.
EBA_RESET	Any 256-byte aligned address	0	Specifies the reset value of the EBA CSR.

Table 16: Verilog Configuration Options (Continued)

Parameter Name	Values	Default	Description
DEBA_RESET	Any 256-byte aligned address	0	Specifies the reset value of the DEB_CSR.
EBR_POSEDGE_REGISTER_FILE	TRUE, FALSE	FALSE	Use EBR to implement register file instead of distributed RAM (LUTs).

EBR Use

The following details of embedded block RAM (EBR) use with different configurations are based on the LatticeECP family of FPGAs.

- ◆ Software-based debugging (DEBUG_ENABLED) requires two EBRs.
- ◆ The instruction and data caches (ICACHE_ENABLED and DCACHE_ENABLED, respectively) require EBR based on the size of the cache:

$$\text{cache size} = \text{sets} \times \text{bytes per cache line} \times \text{associativity}$$

$$\text{number of EBR} = \text{cache size}/1024 + 1$$

For example, the default LatticeMico32 processor in the MSB has software-based debugging, an instruction cache, and a data cache. Both caches have 512 sets, 16 bytes per cache line, and an associativity of 1.

For each cache:

$$\text{cache size} = 512 \times 16 \times 1 = 8192$$

$$\text{number of EBR} = 8192/1024 + 1 = 9$$

Total number of EBRs required:

Software-based debugging	2
Instruction cache	9
Data cache	9
	20

WISHBONE Interconnect Architecture

This chapter describes the standard WISHBONE interconnect architecture that is employed by LatticeMico32 System. It focuses on the items that you must be aware of to begin designing and programming the functions of your system interconnects.

Introduction to WISHBONE Interconnect

LatticeMico32 System uses a standard WISHBONE interconnect architecture to connect the processor to its on-chip component resources such as the LatticeMico32 UART and the LatticeMico32 SPI.

The WISHBONE interconnect works as a general purpose interface, defining the standard data exchanges between the processor module and its components. The interconnect does not interfere with the regulation of the processor or component application-specific functions. Like microcomputer buses, the WISHBONE bus is flexible enough to be tailored to a specific application, robust enough to provide a number of bus cycles and data path widths to solve various system issues, and universal enough to allow a number of suppliers to create design products for it making it more cost-effective.

For more information on the WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores, as it is formally known, refer to the OPENCORES.ORG Web site at:

www.opencores.org/projects.cgi/web/wishbone

The subject matter is very in-depth and goes beyond the scope of this manual.

WISHBONE Registered Feedback Mode

This section describes the WISHBONE Registered Feedback mode. To implement an advanced synchronous cycle termination scheme, Registered Feedback mode bus cycles use the Cycle Type Identifier, CTI_O() and CTI_I(), address tags. Both master and slave interfaces support CTI_O() and CTI_I() for improved bandwidth. The type of burst information is provided by the Burst Type Extension, BTE_O() and BTE_I() address tags.

All WISHBONE Registered Feedback compatible cores must support WISHBONE Classic bus cycles.

Design new IP cores to support WISHBONE Registered Feedback bus cycles to ensure maximum throughput in all systems.

CTI_IO()

The Cycle Type Identifier CTI_IO() address tag provides additional information about the current cycle. The master sends this information to the slave. The slave can use this information to prepare the response for the next cycle.

Table 17: Cycle Type Identifiers

CTI_O(2:0)	Description
000	Classic cycle
001	Constant address burst cycle
010	Incrementing burst cycle
011	Reserved
100	Reserved
101	Reserved
110	Reserved
111	End of burst

Observe the following allowances and rules:

- ◆ Master and slave interfaces may be designed to support the CTI_I() and CTI_O() signals. Also, master and slave interfaces may be designed to support a limited number of burst types.
- ◆ Master and slave interfaces that do support the CTI_I() and CTI_O() signals must at least support the Classic cycle CTI_IO()=000 and the End-of-Cycle CTI_IO()=111.
- ◆ Master and slave interfaces that are designed to support a limited number of burst types must complete the unsupported cycles as though they were WISHBONE Classic cycle, that is, CTI_IO()=000.

- ◆ For description languages that allow default values for input ports (like VHDL), CTI_I() may be assigned a default value of 000.
- ◆ In addition to the WISHBONE Classic rules for generating cycle termination signals ACK_O, RTY_O, and ERR_O, a SLAVE may assert a termination cycle without checking the STB_I signal.
- ◆ ACK_O, RTY_O, and ERR_O may be asserted while STB_O is negated.
- ◆ A cycle terminates when the cycle termination signal, STB_I and STB_O are asserted. Even if ACK_O/ACK_I is asserted, the other signals are only valid when STB_O/STB_I is also asserted.

To avoid the inherent wait state in synchronous termination schemes, the slave must generate the response as soon as possible, that is, the next cycle. It can use the CTI_I() signals to determine the response for the next cycle, but if it cannot determine the state of STB_I for the next cycle, it must generate the response independent of STB_I.

BTE_IO()

The Burst Type Extension BTE_IO() address tag provides additional information about the current burst. The master sends this information to the slave. This information is only relevant for incrementing bursts. In the future, other burst types may use these signals. See Table 18 for BTE_IO(1:0) signal incrementing and decrementing bursts.

Table 18: Burst Type Extension Signal Bursts

BTE_IO(1:0)	Description
00	Linear burst
01	4-beat wrap burst
10	8-beat wrap burst
11	16-beat wrap burst

Observe the following allowances and rules:

- ◆ Master and slave interfaces that support incrementing burst cycles must support the BTE_O() and BTE_I() signals.
- ◆ Master and slave interfaces may be designed to support a limited number of burst extensions.
- ◆ Master and slave interfaces that are designed to support a limited number of burst extensions must complete the unsupported cycles as though they were WISHBONE Classic cycle, that is, CTI_IO() = 000.

Component Signals

In MicoSystem Builder (MSB) you define which components are in the platform and what needs to communicate with what. When the platform generator is run in MSB, it uses this information to build the WISHBONE-based interconnect of the platform. This generated interconnect is a set of Verilog wires connecting the various processor and component ports. To do this, the components must implement certain ports, and follow a specific port naming convention.

Table 19 defines the suffixes that must be used on the names of a component's ports. The suffixes for the ports of a master port are different than the ports of a slave port. The generated interconnect creates signals with names which end with the same suffix as the component port to which the signal is attached.

The prefixes used in the port and signal naming are not described in this section.

Keeping this naming convention in mind, the port/signal descriptions that follow refer to the port/signal which ends with the string in the title.

Table 19: List of Component Port and Signal Name Suffixes

Master Ports		Slave Ports	
Name	Width	Name	Width
_ADR_O	32 bits	_ADR_I	32 bits
_DAT_O	32 bits	_DAT_I	32 bits
_DAT_I	32 bits	_DAT_O	32 bits
_SEL_O	4 bits	_SEL_I	4 bits
_WE_O	1 bit	_WE_I	1 bit
_ACK_I	1 bit	_ACK_O	1 bit
_ERR_I	1 bit	_ERR_O	1 bit
_RTY_I	1 bit	_RTY_O	1 bit
_CTI_O	3 bits	_CTI_I	3 bits
_BTE_O	2 bits	_BTE_I	2 bits
_LOCK_O	1 bit	_LOCK_I	1 bit
_CYC_O	1 bit	_CYC_I	1 bit
_STB_O	1 bit	_STB_I	1 bit

Master Port/Signal Descriptions

This section describes the master ports/signals listed in Table 19.

ADR_O [31:2]

This Address Output array ADR_O() is used to pass a binary address. ADR_O() actually has a full 32 bits. But, because all addressing is on DWORD (4-byte) boundaries, the lowest two bits are always zero.

DAT_O [31:0]

This Data Output array DAT_O() is used to store a binary value for output.

DAT_I [31:0]

This Data Input array DAT_I() is used to store a binary value for input.

SEL_O [3:0]

The Select Output array SEL_O() indicates where valid data is expected on the DAT_I() signal array during READ cycles and where it is placed on the DAT_O() signal array during WRITE cycles. The array boundaries are determined by the granularity of a port.

WE_O

The Write Enable Output WE_O indicates whether the current local bus cycle is a READ or WRITE cycle. The signal is negated during READ cycles and is asserted during WRITE cycles.

ACK_I

This signal is called the Acknowledge Input ACK_I. When asserted, the signal indicates the normal termination of a bus cycle by the slave. Also see the ERR_I and RTY_I signal descriptions.

ERR_I

The Error Input ERR_I indicates an abnormal cycle termination by the slave. The source of the error, and the response generated by the master is defined by the IP core supplier. Also see the ACK_I and RTY_I signal descriptions.

RTY_I

The Retry Input RTY_I indicates that the interface is not ready to accept or send data, and that the cycle should be retried. When and how the cycle is retried is defined by the IP core supplier. Also see the ERR_I and RTY_I signal descriptions.

CTI_O [2:0]

For descriptions of Cycle Type Identifier CTI_O(), see "CTI_IO()" on page 36.

BTE_O [1:0]

For descriptions of Burst Type Extension BTE_O() see "BTE_IO()" on page 37.

LOCK_O

The Lock Output LOCK_O, when asserted, indicates that the current bus cycle is uninterruptible. Lock is asserted to request complete ownership of the bus. Once the transfer has started, the INTERCON does not grant the bus to any other master, until the current master negates LOCK_O or CYC_O.

CYC_O

The Cycle Output CYC_O, when asserted, indicates that a valid bus cycle is in progress. The signal is asserted for the duration of all bus cycles. For example, during a BLOCK transfer cycle there can be multiple data transfers. The CYC_O signal is asserted during the first data transfer and remains asserted until the last data transfer. The CYC_O signal is useful for interfaces with multi-port interfaces, such as dual port memories. In these cases, the CYC_O signal requests use of a common bus from an arbiter.

STB_O

The Strobe Output STB_O indicates a valid data transfer cycle. It is used to qualify various other signals on the interface, such as SEL_O(). The slave asserts either the ACK_I, ERR_I, or RTY_I signals in response to every assertion of the STB_O signal.

Slave Port/Signal Descriptions

This section describes the slave ports/signals listed in the Table 19.

ADR_I [31:2]

The Address Input array ADR_I() is used to pass a binary address. ADR_I() actually has a full 32 bits. But, because all addressing is on DWORD (4-byte) boundaries, the lowest two bits are always zero.

DAT_I [31:0]

This Data Input array DAT_I() is used to store a binary value for input.

DAT_O [31:0]

This Data Output array DAT_O() is used to store a binary value for output.

SEL_I [3:0]

The Select Input array SEL_I() indicates where valid data is placed on the DAT_I() signal array during WRITE cycles and where it should be present on the DAT_O() signal array during READ cycles. The array boundaries are determined by the granularity of a port.

WE_I

The Write Enable Input WE_I indicates whether the current local bus cycle is a READ or WRITE cycle. The signal is negated during READ cycles and is asserted during WRITE cycles.

ACK_O

The Acknowledge Output ACK_O, when asserted, indicates the termination of a normal bus cycle by the slave. Also see the ERR_O and RTY_O signal descriptions.

ERR_O

The Error Output ERR_O indicates an abnormal cycle termination by the slave. The source of the error and the response generated by the master is defined by the IP core supplier. Also see the ACK_O and RTY_O signal descriptions.

RTY_O

The Retry Output RTY_O indicates that the slave interface is not ready to accept or send data and that the cycle should be retried. When and how the cycle is retried is defined by the IP core supplier. Also see the ERR_O and RTY_O signal descriptions.

CTI_I

For descriptions of Cycle Type Identifier CTI_I(), see "CTI_IO()" on page 36.

BTE_I [1:0]

For descriptions of Burst Type Extension BTE_i() "BTE_IO()" on page 37.

LOCK_I

The Lock Input LOCK_I, when asserted, indicates that the current bus cycle is uninterruptible. A slave that receives the LOCK LOCK_I signal is accessed by a single master only until either LOCK_I or CYC_I is negated.

CYC_I [2:0]

The Cycle Input CYC_I, when asserted, indicates that a valid bus cycle is in progress. The signal is asserted for the duration of all bus cycles. For example, during a BLOCK transfer cycle there can be multiple data transfers. The CYC_I signal is asserted during the first data transfer and remains asserted until the last data transfer.

STB_I

The Strobe Input STB_I, when asserted, indicates a valid data transfer cycle. A slave responds to other WISHBONE signals only when this STB_I is asserted, (except for the RST_I signal which should always be responded to). The slave asserts either the ACK_O, ERR_O, or RTY_O signals in response to every assertion of the STB_I signal.

Opcode Look-Up Table

Opcode	Decimal	Hexadecimal	Mnemonic	Opcode	Decimal	Hexadecimal	Mnemonic
000000	00	00	srui	100000	32	20	sru
000001	01	01	nori	100001	33	21	nor
000010	02	02	muli	100010	34	22	mul
000011	03	03	sh	100011	35	23	divu
000100	04	04	lb	100100	36	24	rcsr
000101	05	05	sri	100101	37	25	sr
000110	06	06	xori	100110	38	26	xor
000111	07	07	lh	100111	39	27	div
001000	08	08	andi	101000	40	28	and
001001	09	09	xnori	101001	41	29	xnor
001010	10	0A	lw	101010	42	2A	reserved
001011	11	0B	lhu	101011	43	2B	raise
001100	12	0C	sb	101100	44	2C	sextb
001101	13	0D	addi	101101	45	2D	add
001110	14	0E	ori	101110	46	2E	or
001111	15	0F	sli	101111	47	2F	sl
010000	16	10	lbu	110000	48	30	b
010001	17	11	be	110001	49	31	modu
010010	18	12	bg	110010	50	32	sub
010011	19	13	bge	110011	51	33	reserved
010100	20	14	bgeu	110100	52	34	wcsr
010101	21	15	bgu	110101	53	35	mod
010110	22	16	sw	110110	54	36	call
010111	23	17	bne	110111	55	37	sexth
011000	24	18	andhi	111000	56	38	bi
011001	25	19	cmpei	111001	57	39	cmpe
011010	26	1A	cmpgi	111010	58	3A	cmpg
011011	27	1B	cmpgei	111011	59	3B	cmpge
011100	28	1C	cmpgeui	111100	60	3C	cmpgeu
011101	29	1D	cmpgui	111101	61	3D	cmpgu
011110	30	1E	orhi	111110	62	3E	calli
011111	31	1F	cmpnei	111111	63	3F	cmpne

Pseudo-Instructions

To aid the semantics of assembler programs, the LatticeMico32 assembler implements a variety of pseudo-instructions. Table 20 lists these instructions and to what actual instructions they are mapped. Disassemblers show the actual implementation.

Table 20: Pseudo-Instructions

Mnemonic	Implementation	Description
ret	b ra	Return from function call.
mv rX, rY	or rX, rY, r0	Move value in rY to rX.
mvhi rX, imm16	orhi rX, r0, imm16	Move the 16-bit, left-shifted immediate into rX.
not rX, rY	xnor rX, rY, r0	Bitwise complement of the value in rY, storing the result in rX.

Instruction Descriptions

add

Figure 24: add Instruction

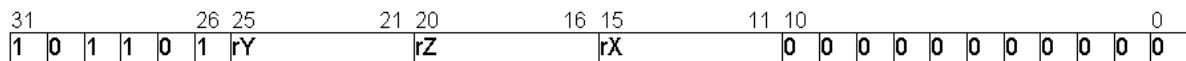


Table 21: add Instruction Features

Feature	Description
Operation	Integer addition
Description	Adds the value in rY to the value in rZ, storing the result in rX.
Syntax	add rX, rY, rZ
Example	add r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] + \text{gpr}[\text{rZ}]$
Result	1 cycle
Issue	1 cycle
See Also	addi, addition with immediate

addi

Figure 25: addi Instruction

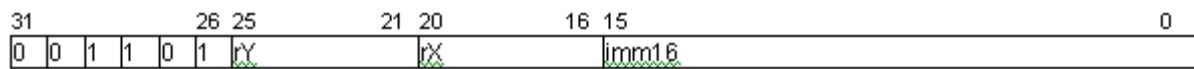


Table 22: addi Instruction Features

Feature	Description
Operation	Integer addition with immediate
Description	Adds the value in rY to the sign-extended immediate, storing the result in rX.
Syntax	<code>addi rX, rY, imm16</code>
Example	<code>addi r4, r2, -32</code>
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] + \text{sign_extend}(\text{imm16})$
Result	1 cycle
Issue	1 cycle
See Also	add, addition between registers

and

Figure 26: and Instruction

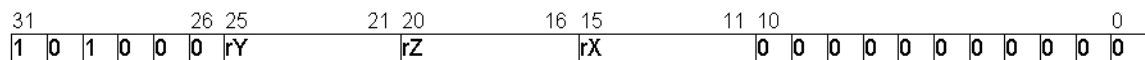


Table 23: and Instruction Features

Feature	Description
Operation	Bitwise logical AND
Description	Bitwise AND of the value in rY with the value in rZ, storing the result in rX.
Syntax	and rX, rY, rZ
Example	and r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \& \text{gpr}[\text{rZ}]$
Result	1 cycle
Issue	1 cycle
See Also	andi, AND with immediate; andhi, AND with high 16 bits

andhi

Figure 27: andhi Instruction

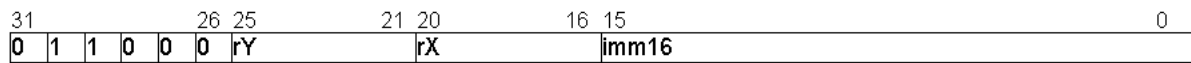


Table 24: andhi Instruction Features

Feature	Description
Operation	Bitwise logical AND (high 16-bits)
Description	Bitwise AND of the value in rY with the 16-bit, left-shifted immediate, storing the result in rX.
Syntax	andhi rX, rY, imm16
Example	andhi r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \& (\text{imm16} \ll 16)$
Result	1 cycle
Issue	1 cycle
See Also	AND between registers; andi, AND with immediate

andi

Figure 28: andi Instruction

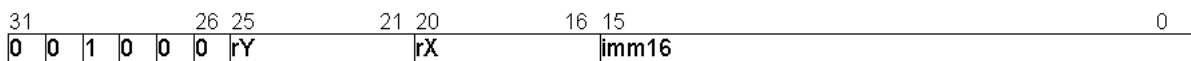
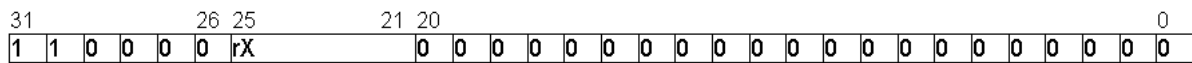
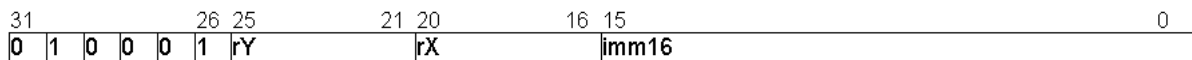


Table 25: andi Instruction Features

Feature	Description
Operation	Bitwise logical AND
Description	Bitwise AND of the value in rY with the zero-extended immediate, storing the result in rX.
Syntax	andi rX, rY, imm16
Example	andi r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \& \text{zero_extend}(\text{imm16})$
Result	1 cycle
Issue	1 cycle
See Also	and, AND between registers; andhi, AND with high 16 bits

b**Figure 29: b Instruction****Table 26: b Instruction Features**

Feature	Description
Operation	Unconditional branch
Description	Unconditional branch to address in rX. rX cannot be r30 (ea) or r31 (ba).
Syntax	b rX
Example	b r3
Semantics	PC = gpr[rX]
Issue	4 cycles
See Also	bi, branch with immediate

be**Figure 30: be Instruction****Table 27: be Instruction Features**

Feature	Description
Operation	Branch if equal
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the values are equal.
Syntax	be rX, rY, imm16
Example	be r4, r2, label
Semantics	if (gpr[rX] == gpr[rY]) PC = PC + sign_extend(imm16 << 2)
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	bne, branch if not equal

bg

Figure 31: bg Instruction

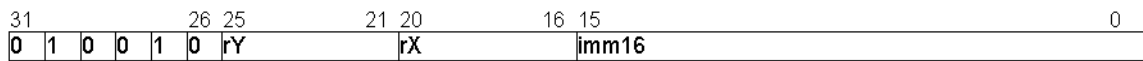


Table 28: bg Instruction Features

Feature	Description
Operation	Branch if greater
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the value in rX is greater than the value in rY. The values in rX and rY are treated as signed integers.
Syntax	bg rX, rY, imm16
Example	bg r4, r2, label
Semantics	<pre>if (gpr[rX] > gpr[rY]) PC = PC + sign_extend(imm16 << 2)</pre>
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	bgu, branch if greater, unsigned

bge

Figure 32: bge Instruction

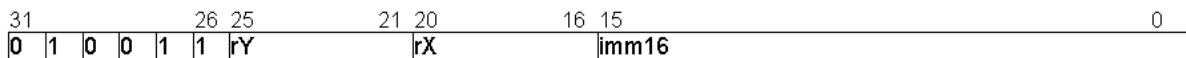


Table 29: bge Instruction Features

Feature	Description
Operation	Branch if greater or equal
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the value in rX is greater or equal to the value in rY. The values in rX and rY are treated as signed integers.
Syntax	bge rX, rY, imm16
Example	bge r4, r2, label
Semantics	<pre>if (gpr[rX] >= gpr[rY]) PC = PC + sign_extend(imm16 << 2)</pre>
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	bgeu, branch if greater or equal, unsigned

bgeu

Figure 33: bgeu Instruction

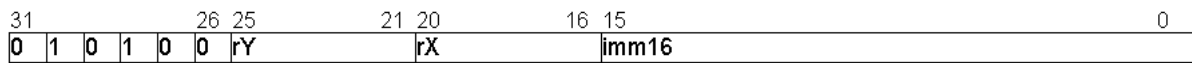


Table 30: bgeu Instruction Features

Feature	Description
Operation	Branch if greater or equal, unsigned
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the value in rX is greater or equal to the value in rY. The values in rX and rY are treated as unsigned integers.
Syntax	bgeu rX, rY, imm16
Example	bgeu r4, r2, label
Semantics	if (gpr[rX] >= gpr[rY]) PC = PC + sign_extend(imm16 << 2)
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	bge, branch if greater or equal, signed

bgu

Figure 34: bgu Instruction

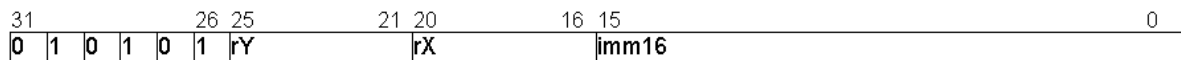


Table 31: bgu Instruction Features

Feature	Description
Operation	Branch if greater, unsigned
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the value in rX is greater than the value in rY. The values in rX and rY are treated as unsigned integers.
Syntax	bgu rX, rY, imm16
Example	bgu r4, r2, label
Semantics	if (gpr[rX] > gpr[rY]) PC = PC + sign_extend(imm16 << 2)
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	bg, branch if greater, signed

bi

Figure 35: bi Instruction

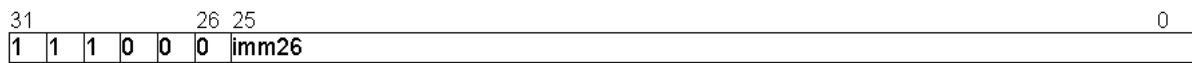


Table 32: bi Instruction Features

Feature	Description
Operation	Unconditional branch
Description	Unconditional branch to the address given by the sum of the PC and the sign-extended immediate.
Syntax	bi imm26
Example	bi label
Semantics	PC = PC + sign_extend(imm26 << 2)
Issue	4 cycles
See Also	b, branch from register

bne

Figure 36: bne Instruction

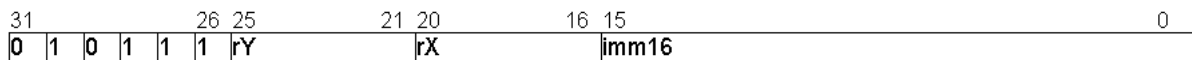


Table 33: bne Instruction Features

Feature	Description
Operation	Branch if not equal
Description	Compares the value in rX with the value in rY, branching to the address given by the sum of the PC and the sign-extended immediate if the values are not equal.
Syntax	bne rX, rY, imm16
Example	bne r4, r2, label
Semantics	if (gpr[rX] != gpr[rY]) PC = PC + sign_extend(imm16 << 2)
Issue	1 cycle (not-taken), 4 cycles (taken)
See Also	be, branch if equal

break

Figure 37: break Instruction

31	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Table 34: break Instruction Features

Feature	Description
Operation	Software breakpoint
Description	Raises a breakpoint exception.
Syntax	<code>break</code>
Example	<code>break</code>
Semantics	$\text{gpr}[\text{ba}] = \text{PC}$ $\text{IE.BIE} = \text{IE.IE}$ $\text{IE.IE} = 0$ $\text{PC} = \text{DEBA} + \text{ID} * 32$
Issue	4 cycles
See Also	<code>bret</code> , return from breakpoint

bret

Figure 38: bret Instruction

31	1	1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
----	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Table 35: bret Instruction Features

Feature	Description
Operation	Return from breakpoint
Description	Unconditional branch to the address in the breakpoint address register (ba), updating interrupt enable with value saved in breakpoint interrupt enable register.
Syntax	<code>bret</code>
Example	<code>bret</code>
Semantics	$\text{PC} = \text{gpr}[\text{ba}]$ $\text{IE.IE} = \text{IE.BIE}$
Issue	4 cycles
See Also	<code>break</code> , breakpoint

call

Figure 39: call Instruction

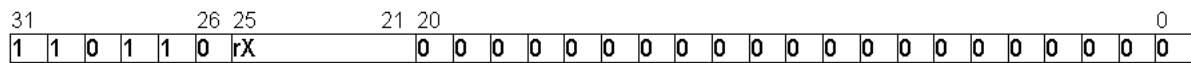


Table 36: call Instruction Features

Feature	Description
Operation	Function call
Description	Adds 4 to the PC, storing the result in ra, then unconditionally branches to the address in rX.
Syntax	<code>call rX</code>
Example	<code>call r3</code>
Semantics	$\text{gpr}[\text{ra}] = \text{PC} + 4$ $\text{PC} = \text{gpr}[\text{rX}]$
Result	1 cycle
Issue	4 cycles
See Also	calli, call with immediate; ret, return from call

calli

Figure 40: calli Instruction



Table 37: calli Instruction Features

Feature	Description
Operation	Function call
Description	Adds 4 to the PC, storing the result in ra, then unconditionally branches to the address given by the sum of the PC and the sign-extended immediate.
Syntax	<code>calli imm26</code>
Example	<code>calli label</code>
Semantics	$\text{gpr}[\text{ra}] = \text{PC} + 4$ $\text{PC} = \text{PC} + \text{sign_extend}(\text{imm26} \ll 2)$
Result	1 cycle
Issue	4 cycles
Latency	4 cycles
See Also	call, call from register; ret, return from call

cmpe

Figure 41: cmpe Instruction

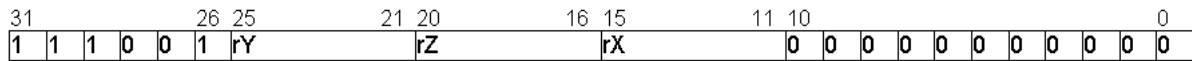


Table 38: cmpe Instruction Features

Feature	Description
Operation	Compare equal
Description	Compares the value in rY with the value in rZ, storing 1 in rX if they are equal, otherwise 0.
Syntax	cmpe rX, rY, rZ
Example	cmpe r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] == \text{gpr}[\text{rZ}]$
Result	2 cycles
Issue	1 cycle
See Also	cmpei, compare equal with immediate

cmpei

Figure 42: cmpei Instruction

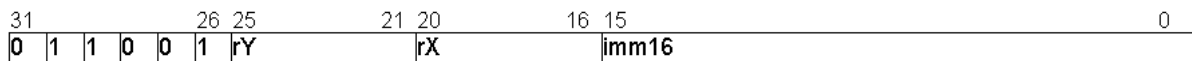


Table 39: cmpei Instruction Features

Feature	Description
Operation	Compare equal
Description	Compares the value in rY with the sign-extended immediate, storing 1 in rX if they are equal, 0 otherwise.
Syntax	cmpei rX, rY, imm16
Example	cmpei r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] == \text{sign_extend}(\text{imm16})$
Result	2 cycles
Issue	1 cycle
See Also	cmpe, compare equal between registers

cmpg

Figure 43: cmpg Instruction

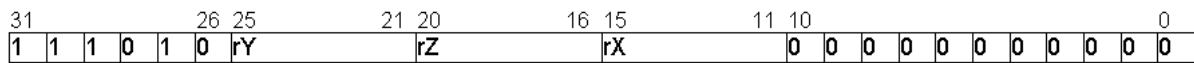


Table 40: cmpg Instruction Features

Feature	Description
Operation	Compare greater
Description	Compares the value in rY with the value in rZ, storing 1 in rX if the value in rY is greater than the value in rZ, 0 otherwise. Both operands are treated as signed integers.
Syntax	<code>cmpg rX, rY, rZ</code>
Example	<code>cmpg r14, r15, r17</code>
Semantics	<code>gpr[rX] = gpr[rY] > gpr[rZ]</code>
Result	2 cycles
Issue	1 cycle
See Also	<code>cmpgi</code> , compare greater with immediate; <code>cmpgu</code> , compare greater, unsigned; <code>cmpgui</code> , compare greater with immediate, unsigned

cmpgi

Figure 44: cmpgi Instruction

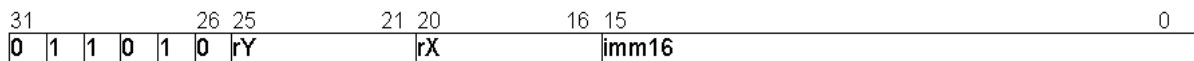


Table 41: cmpgi Instruction Features

Feature	Description
Operation	Compare greater
Description	Compares the value in rY with the sign-extended immediate, storing 1 in rX if the value in rY is greater than the immediate, 0 otherwise. Both operands are treated as signed integers.
Syntax	<code>cmpgi rX, rY, imm16</code>
Example	<code>cmpgi r4, r2, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[rY] > sign_extend(imm16)</code>
Result	2 cycles
Issue	1 cycle
See Also	<code>cmpg</code> , compare greater between registers; <code>cmpgu</code> , compare greater, unsigned; <code>cmpgui</code> , compare greater with immediate, unsigned

cmpge

Figure 45: cmpge Instruction

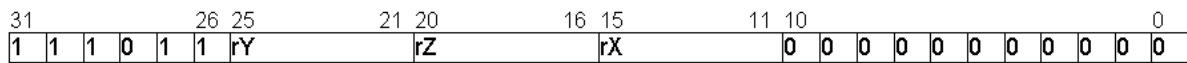


Table 42: cmpge Instruction Features

Feature	Description
Operation	Compare greater or equal
Description	Compares the value in rY with the value in rZ, storing 1 in rX if the value in rY is greater or equal to the value in rZ, 0 otherwise. Both operands are treated as signed integers.
Syntax	cmpge rX, rY, rZ
Example	cmpge r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \geq \text{gpr}[\text{rZ}]$
Result	2 cycles
Issue	1 cycle
See Also	cmpgei, compare with immediate; cmpgeu, compare, unsigned; cmpgeui, compare with immediate, unsigned

cmpgei

Figure 46: cmpgei Instruction

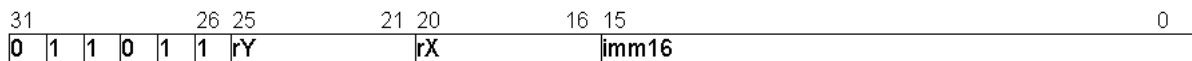


Table 43: cmpgei Instruction Features

Feature	Description
Operation	Compare greater or equal
Description	Compares the value in rY with the sign-extended immediate, storing 1 in rX if the value in rY is greater or equal to the immediate, 0 otherwise. Both operands are treated as signed integers.
Syntax	cmpgei rX, rY, imm16
Example	cmpgei r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \geq \text{sign_extend}(\text{imm16})$
Result	2 cycles
Issue	1 cycle
See Also	cmpge, compare between registers; cmpgeu, compare, unsigned; cmpgeui, compare with immediate, unsigned

cmpgeu

Figure 47: cmpgeu Instruction

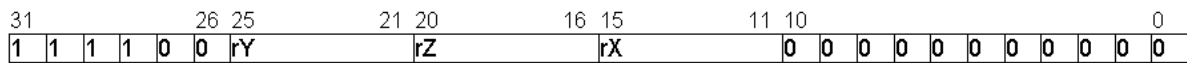


Table 44: cmpgeu Instruction Features

Feature	Description
Operation	Compare greater or equal
Description	Compares the value in rY with the value in rZ, storing 1 in rX if the value in rY is greater or equal to the value in rZ, 0 otherwise. Both operands are treated as unsigned integers.
Syntax	<code>cmpgeu rX, rY, rZ</code>
Example	<code>cmpgeu r14, r15, r17</code>
Semantics	<code>gpr[rX] = gpr[rY] >= gpr[rZ]</code>
Result	2 cycles
Issue	1 cycle
See Also	<code>cmpge</code> , compare between registers; <code>cmpgei</code> , compare with immediate; <code>cmpgeui</code> , compare with immediate, unsigned

cmpgeui

Figure 48: cmpgeui Instruction

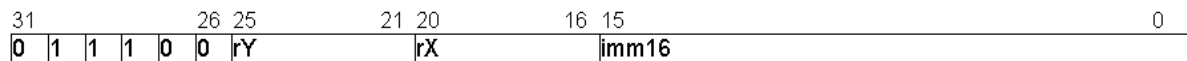


Table 45: cmpgeui Instruction Features

Feature	Description
Operation	Compare greater or equal
Description	Compares the value in rY with the zero-extended immediate, storing 1 in rX if the value in rY is greater or equal to the immediate, 0 otherwise. Both operands are treated as unsigned integers.
Syntax	<code>cmpgeui rX, rY, imm16</code>
Example	<code>cmpgeui r4, r2, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[rY] >= zero_extend(imm16)</code>
Result	2 cycles
Issue	1 cycle
See Also	<code>cmpge</code> , compare between registers; <code>cmpgei</code> , compare with immediate; <code>cmpgeu</code> , compare, unsigned

cmpgu

Figure 49: cmpgu Instruction

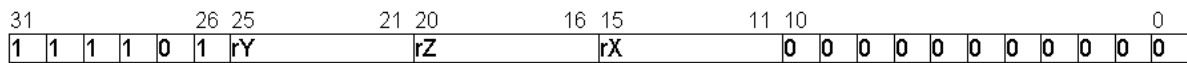


Table 46: cmpgu Instruction Features

Feature	Description
Operation	Compare greater unsigned
Description	Compares the value in rY with the value in rZ, storing 1 in rX if the value in rY is greater than the value in rZ, 0 otherwise. Both operands are treated as unsigned integers.
Syntax	cmpgu rX, rY, rZ
Example	cmpgu r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] > \text{gpr}[\text{rZ}]$
Result	2 cycles
Issue	1 cycle
See Also	cmpg, compare greater, signed; cmpgi, compare greater with immediate; cmpgui, compare greater with immediate, unsigned

cmpgui

Figure 50: cmpgui Instruction

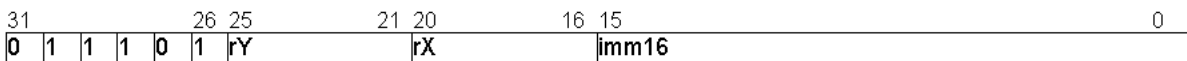


Table 47: cmpgui Instruction Features

Feature	Description
Operation	Compare greater unsigned
Description	Compares the value in rY with the zero-extended immediate, storing 1 in rX if the value in rY is greater than the immediate, 0 otherwise. Both operands are treated as unsigned integers.
Syntax	<code>cmpgui rX, rY, imm16</code>
Example	<code>cmpgui r4, r2, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[rY] > zero_extend(imm16)</code>
Result	2 cycles
Issue	1 cycle
See Also	<code>cmpg</code> , compare greater, signed; <code>cmpgi</code> , compare greater with immediate; <code>cmpgu</code> , compare greater, unsigned

cmpne

Figure 51: cmpne Instruction

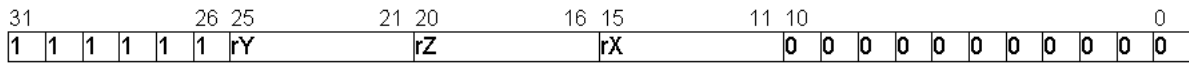


Table 48: cmpne Instruction Features

Feature	Description
Operation	Compare not equal
Description	Compares the value in rY with the value in rZ, storing 1 in rX if they are not equal, 0 otherwise.
Syntax	cmpne rX, rY, rZ
Example	cmpne r14, r15, r17
Semantics	$\text{gpr}[rX] = \text{gpr}[rY] \neq \text{gpr}[rZ]$
Result	2 cycles
Issue	1 cycle
See Also	cmpnei, compare not equal with immediate

cmpnei

Figure 52: cmpnei Instruction

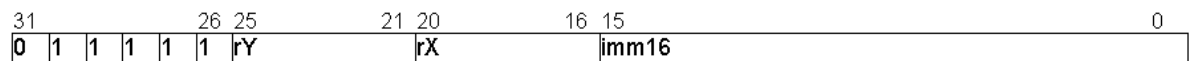


Table 49: cmpnei Instruction Features

Feature	Description
Operation	Compare not equal
Description	Compares the value in rY with the sign-extended immediate, storing 1 in rX if they are not equal, 0 otherwise.
Syntax	cmpnei rX, rY, imm16
Example	cmpnei r4, r2, 0x5555
Semantics	$\text{gpr}[rX] = \text{gpr}[rY] \neq \text{sign_extend}(\text{imm16})$
Result	2 cycles
Issue	1 cycle
See Also	cmpne, compare not equal between registers

divu

Figure 53: divu Instruction

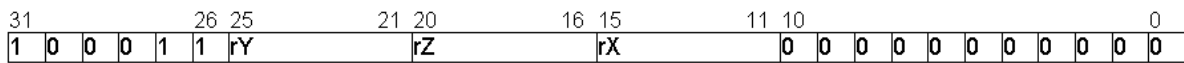


Table 50: divu Instruction Features

Feature	Description
Operation	Unsigned iinteger division
Description	Divides the value in rY by the value in rZ, storing the quotient in rX. Both operands are treated as unsigned integers. Available only if the processor was configured with the DIVIDE_ENABLED option.
Syntax	divu rX, rY, rZ
Example	divu r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] / \text{gpr}[\text{rZ}]$
Result	34 cycles
Issue	34 cycles
See Also	modu, modulus

eret

Figure 54: eret Instruction

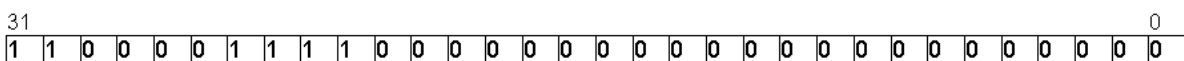


Table 51: eret Instruction Features

Feature	Description
Operation	Return from exception
Description	Unconditionally branches to the address in the exception address register (ea), updating interrupt enable with value saved in exception interrupt enable register.
Syntax	eret
Example	eret
Semantics	$\text{PC} = \text{gpr}[\text{ea}]$ $\text{IE.IE} = \text{IE.EIE}$
Result	
Issue	3 cycles
See Also	scall, system call

lh

Figure 57: lh Instruction

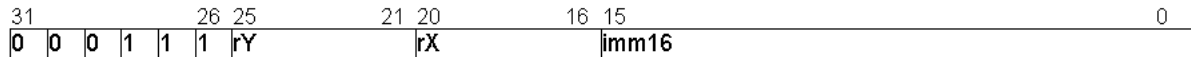


Table 54: lh Instruction Features

Feature	Description
Operation	Load half-word from memory
Description	Loads a half-word from memory at the address specified by the sum of the value in rY added to the sign-extended immediate, storing the sign-extended result into rX.
Syntax	lh rX, (rY+imm16)
Example	lh r4, (r2+6)
Semantics	<pre>address = gpr[rY] + sign_extend(imm16) gpr[rX] = sign_extend((memory[address] << 8) (memory[address+1]))</pre>
Result	3 cycles
Issue	1 cycle
See Also	lb, load byte, signed; lbu, load byte, unsigned; lhu, load half-word, unsigned; lw, load word

lhu

Figure 58: lhu Instruction

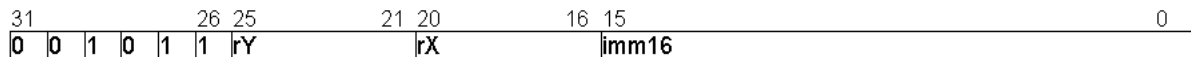


Table 55: lhu Instruction Features

Feature	Description
Operation	Load unsigned half-word from memory
Description	Loads a half-word from memory at the address specified by the sum of the value in rY added to the sign-extended immediate, storing the zero-extended result into rX.
Syntax	lhu rX, (rY+imm16)
Example	lhu r4, (r2+6)
Semantics	<pre>address = gpr[rY] + sign_extend(imm16) gpr[rX] = zero_extend((memory[address] << 8) (memory[address+1]))</pre>
Result	3 cycles
Issue	1 cycle
See Also	lb, load byte, signed; lbu, load byte, unsigned; lh, load half-word, signed; lw, load word

lw

Figure 59: lw Instruction

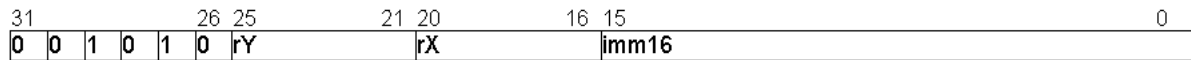


Table 56: lw Instruction Features

Feature	Description
Operation	Load word from memory
Description	Loads a word from memory at address specified by the sum of the value in rY added to the sign-extended immediate, storing the result in rX.
Syntax	<code>lw rX, (rY+imm16)</code>
Example	<code>lw r4, (r2+8)</code>
Semantics	<pre> address = gpr[rY] + sign_extend(imm16) gpr[rX] = (memory[address] << 24) (memory[address+1] << 16) (memory[address+2] << 8) (memory[address+3]) </pre>
Result	3 cycles
Issue	1 cycle
See Also	lb, load byte, signed; lbu, load byte, unsigned; lh, load half-word, signed; lhu, load half-word, unsigned

modu

Figure 60: modu Instruction

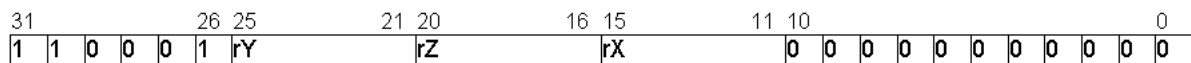


Table 57: modu Instruction Features

Feature	Description
Operation	Unsigned integer modulus
Description	Divides the value in rY by the value in rZ, storing the remainder in rX. Both operands are treated as unsigned integers. Available only if the processor was configured with the DIVIDE_ENABLED option.
Syntax	<code>modu rX, rY, rZ</code>
Example	<code>modu r14, r15, r17</code>
Semantics	<code>gpr[rX] = gpr[rY] % gpr[rZ]</code>
Result	34 cycles
Issue	34 cycles
See Also	<code>divu</code> , <code>divide</code>

mul

Figure 61: mul Instruction

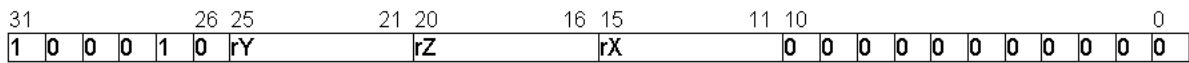


Table 58: mul Instruction Features

Feature	Description
Operation	Integer multiply
Description	Multiplies the value in rY by the value in rZ, storing the low 32 bits of the product in rX. Available only if the processor was configured with either the MC_MULTIPLY_ENABLED or PL_MULTIPLY_ENABLED option.
Syntax	mul rX, rY, rZ
Example	mul r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] * \text{gpr}[\text{rZ}]$
Result	3 cycles
Issue	1 cycle
See Also	muli, multiply with immediate

muli

Figure 62: muli Instruction

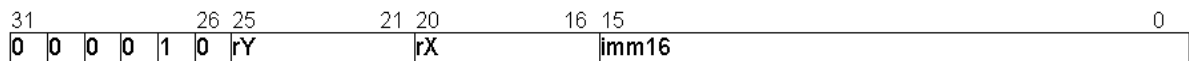


Table 59: muli Instruction Features

Feature	Description
Operation	Integer multiply
Description	Multiplies the value in rY by the sign-extended immediate, storing the low 32 bits of the product in rX. Available only if the processor was configured with either the MC_MULTIPLY_ENABLED or PL_MULTIPLY_ENABLED option.
Syntax	muli rX, rY, imm16
Example	muli r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] * \text{sign_extend}(\text{imm16})$
Result	3 cycles
Issue	1 cycle
See Also	mul, multiply between registers

mv

Feature	Description
Operation	Move
Description	Moves the value in rY to rX. This is a pseudo-instruction implemented with: <code>or rX, rY, r0</code> .
Syntax	<code>mv rX, rY</code>
Example	<code>mv r4, r2</code>
Semantics	<code>gpr[rX] = gpr[rY] gpr[r0]</code>
Result	1 cycle
Issue	1 cycle
See Also	<code>mvhi</code> , move immediate into high 16 bits

mvhi

Feature	Description
Operation	Move high 16 bits
Description	Moves the 16-bit, left-shifted immediate into rX. This is a pseudo-instruction implemented with: <code>orhi rX, r0, imm16</code> .
Syntax	<code>mvhi rX, imm16</code>
Example	<code>mvhi r4, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[r0] (imm16 << 16)</code>
Result	1 cycle
Issue	1 cycle
See Also	<code>mv</code> , move between registers

nor

Figure 63: nor Instruction

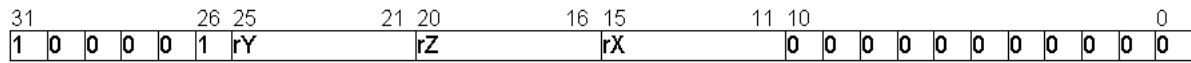


Table 60: nor Instruction Features

Feature	Description
Operation	Bitwise logical NOR
Description	Bitwise NOR of the value in rY with the value in rZ, storing the result in rX.
Syntax	nor rX, rY, rZ
Example	nor r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \sim(\text{gpr}[\text{rY}] \mid \text{gpr}[\text{rZ}])$
Result	1 cycle
Issue	1 cycle
See Also	norl, NOR with immediate

norl

Figure 64: norl Instruction

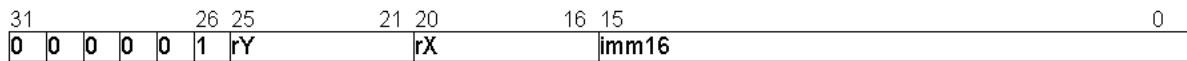


Table 61: norl Instruction Features

Feature	Description
Operation	Bitwise logical NOR
Description	Bitwise NOR of the value in rY with the zero-extended immediate, storing the result in rX.
Syntax	norl rX, rY, imm16
Example	norl r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \sim(\text{gpr}[\text{rY}] \mid \text{zero_extend}(\text{imm16}))$
Result	1 cycle
Issue	1 cycle
See Also	nor, NOR between registers

not

Feature	Description
Operation	Bitwise complement
Description	Bitwise complement of the value in rY, storing the result in rX. This is a pseudo-instruction implemented with: <code>xnor rX, rY, r0</code> .
Syntax	<code>not rX, rY</code>
Example	<code>not r4, r2</code>
Semantics	$\text{gpr}[\text{rX}] = \sim(\text{gpr}[\text{rY}] \wedge \text{gpr}[\text{r0}])$
Result	1 cycle
Issue	1 cycle

or

Figure 65: or Instruction

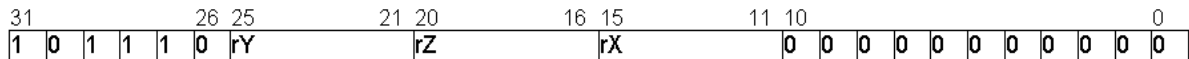


Table 62: or Instruction Features

Feature	Description
Operation	Bitwise logical OR
Description	Bitwise OR of the value in rY with the value in rZ, storing the result in rX.
Syntax	<code>or rX, rY, rZ</code>
Example	<code>or r14, r15, r17</code>
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \mid \text{gpr}[\text{rZ}]$
Result	1 cycle
Issue	1 cycle
See Also	<code>ori</code> , OR with immediate; <code>orhi</code> , OR with high 16 bits

ori

Figure 66: ori Instruction

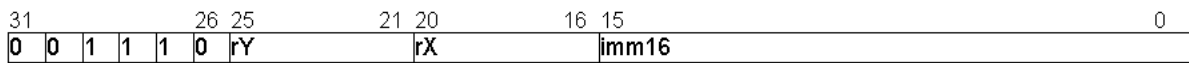


Table 63: ori Instruction Features

Feature	Description
Operation	Bitwise logical OR
Description	Bitwise OR of the value in rY with the zero-extended immediate, storing the result in rX.
Syntax	<code>ori rX, rY, imm16</code>
Example	<code>ori r4, r2, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[rY] zero_extend(imm16)</code>
Result	1 cycle
Issue	1 cycle
See Also	or, OR between registers; orhi, OR with high 16 bits

orhi

Figure 67: orhi Instruction

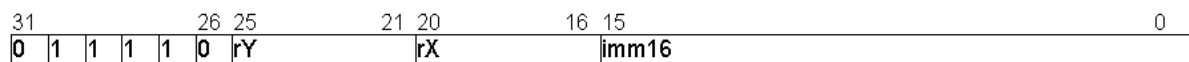


Table 64: orhi Instruction Features

Feature	Description
Operation	Bitwise logical OR (high 16-bits)
Description	Bitwise OR of the value in rY with the 16-bit, left-shifted immediate, storing the result in rX.
Syntax	<code>orhi rX, rY, imm16</code>
Example	<code>orhi r4, r2, 0x5555</code>
Semantics	<code>gpr[rX] = gpr[rY] (imm16 << 16)</code>
Result	1 cycle
Issue	1 cycle
See Also	or, OR between registers; ori, OR with immediate

rcsr

Figure 68: rcsr Instruction

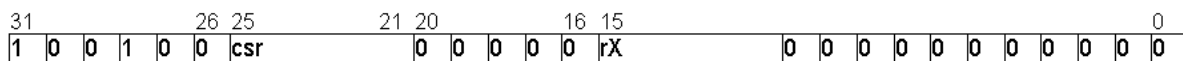


Table 65: rcsr Instruction Features

Feature	Description
Operation	Read control and status register
Description	Reads the value of the specified control and status register and stores it in rX.
Syntax	<code>rcsr rX, csr</code>
Example	<code>rcsr r15, IM</code>
Semantics	<code>gpr[rX] = csr</code>
Result	1 cycle
Issue	1 cycle
See Also	wcsr, write control and status register

ret

Feature	Description
Operation	Return from function call
Description	Unconditional branch to address in ra. This is a pseudo-instruction implemented with: <code>b ra</code> .
Syntax	<code>ret</code>
Example	<code>ret</code>
Semantics	<code>PC = gpr[ra]</code>
Result	
Issue	4 cycles
See Also	call, function call from register; calli, function call with immediate

sb

Figure 69: sb Instruction

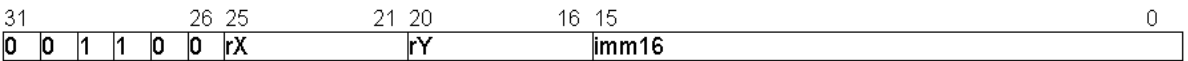


Table 66: sb Instruction Features

Feature	Description
Operation	Store byte to memory
Description	Stores the lower byte in rY into memory at the address specified by the sum of the value in rX added to the sign-extended immediate.
Syntax	sb(rX+imm16), rY
Example	sb(r2+8), r4
Semantics	address = gpr[rX] + sign_extend(imm16) memory[address] = gpr[rY] & 0xff
Result	
Issue	1 cycle
See Also	sh, store half-word; sw, store word

scall

Figure 70: scall Instruction

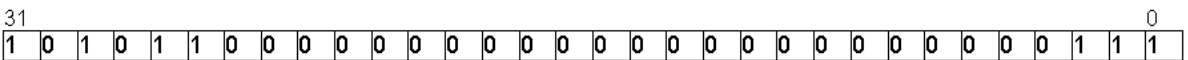


Table 67: scall Instruction Features

Feature	Description
Operation	System call
Description	Raises a system call exception.
Syntax	scall
Example	scall
Semantics	gpr[ea] = PC IE.EIE = IE.IE IE.IE = 0 PC = (DC.RE ? DEBA : EBA) + ID * 32
Result	
Issue	4 cycles
See Also	eret, return from exception

sextb

Figure 71: sextb Instruction

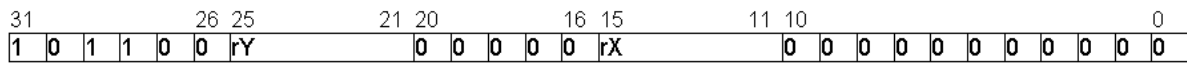


Table 68: sextb Instruction Features

Feature	Description
Operation	Sign-extend byte to word
Description	Sign-extends the value in rY, storing the result in rX. Available only if the processor was configured with the SIGN_EXTEND_ENABLED option.
Syntax	sextb rX, rY
Example	sextb r14, r15
Semantics	$\text{gpr}[\text{rX}] = (\text{gpr}[\text{rY}] \ll 24) \gg 24$
Result	1 cycle
Issue	1 cycle
See Also	sextb, sign-extend half-word

sextb

Figure 72: sextb Instruction

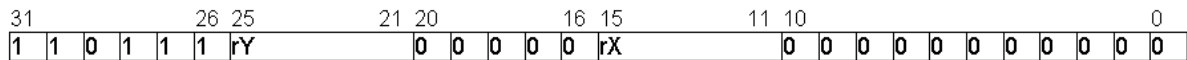


Table 69: sextb Instruction Features

Feature	Description
Operation	Sign-extends half-word to word
Description	Sign-extends the value in rY, storing the result in rX. Available only if the processor was configured with the SIGN_EXTEND_ENABLED option.
Syntax	sextb rX, rY
Example	sextb r14, r15
Semantics	$\text{gpr}[\text{rX}] = (\text{gpr}[\text{rY}] \ll 16) \gg 16$
Result	1 cycle
Issue	1 cycle
See Also	sextb, sign-extend byte

sh

Figure 73: sh Instruction

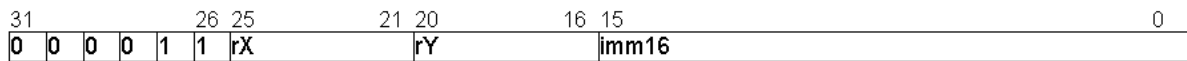


Table 70: sh Instruction Features

Feature	Description
Operation	Store half-word to memory
Description	Stores the lower half-word in rY into memory at the address specified by the sum of the value in rX added to the sign-extended immediate.
Syntax	sh (rX+imm16), rY
Example	sh (r2+8), r4
Semantics	address = gpr[rX] + sign_extend(imm16) memory[address] = gpr[rY] & 0xff memory[address+1] = (gpr[rY] >> 8) & 0xff
Result	
Issue	1 cycle
See Also	sb, store byte; sw, store word

sl

Figure 74: sl Instruction

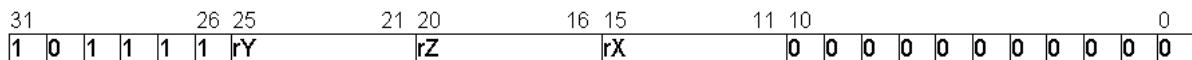


Table 71: sl Instruction Features

Feature	Description
Operation	Shift left
Description	Shifts the value in rY left by the number of bits specified by the value in rZ, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	sl rX, rY, rZ
Example	sl r14, r15, r17
Semantics	$\text{gpr}[rX] = \text{gpr}[rY] \ll (\text{gpr}[rZ] \& 0x1f)$
Result	2 cycles
Issue	1 cycle
See Also	sli, shift left with immediate

sli

Figure 75: sli Instruction

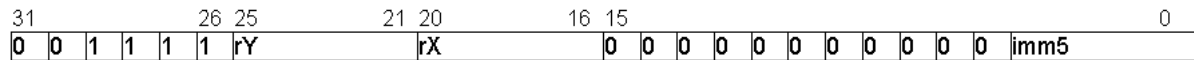


Table 72: sli Instruction Features

Feature	Description
Operation	Shift left
Description	Shifts the value in rY left by the number of bits specified by the immediate, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	<code>sli rX, rY, imm5</code>
Example	<code>sli r4, r2, 17</code>
Semantics	<code>gpr[rX] = gpr[rY] << imm5</code>
Result	2 cycles
Issue	1 cycle
See Also	sl, shift left from register

sr

Figure 76: sr Instruction

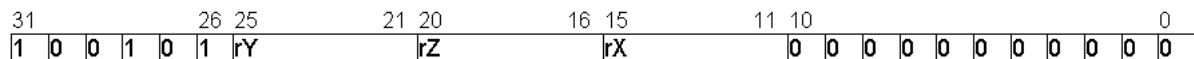


Table 73: sr Instruction Features

Feature	Description
Operation	Shift right (arithmetic)
Description	Shifts the signed value in rY right by the number of bits specified by the value in rZ, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	sr rX, rY, rZ
Example	sr r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \gg (\text{gpr}[\text{rZ}] \& 0\text{x1f})$
Result	2 cycles
Issue	1 cycle
See Also	sri, shift right with immediate; sru, shift right, unsigned; srui, shift right with immediate, unsigned

sri

Figure 77: sri Instruction

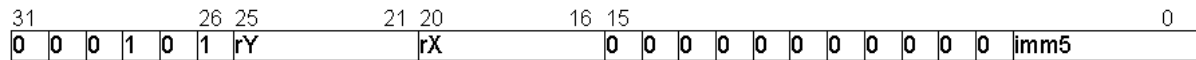


Table 74: sri Instruction Features

Feature	Description
Operation	Shift right (arithmetic)
Description	Shifts the signed value in rY right by the number of bits specified by the immediate, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	<code>sri rX, rY, imm5</code>
Example	<code>sri r4, r2, 12</code>
Semantics	<code>gpr[rX] = gpr[rY] >> imm5</code>
Result	2 cycles
Issue	1 cycle
See Also	sr, shift right from register; sru, shift right, unsigned; srui, shift right with immediate, unsigned

sru

Figure 78: sru Instruction

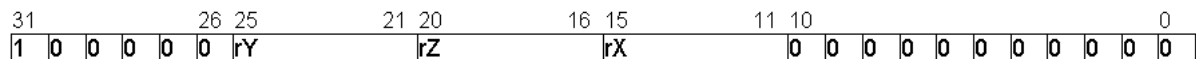


Table 75: sru Instruction Features

Feature	Description
Operation	Shift right, unsigned (logical)
Description	Shifts the unsigned value in rY right by the number of bits specified by the value in rZ, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	sru rX, rY, rZ
Example	sru r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \gg (\text{gpr}[\text{rZ}] \& 0\text{x1f})$
Result	2 cycles
Issue	1 cycle
See Also	sr, shift right from register; sri, shift right with immediate; srui, shift right with immediate, unsigned

srui

Figure 79: srui Instruction

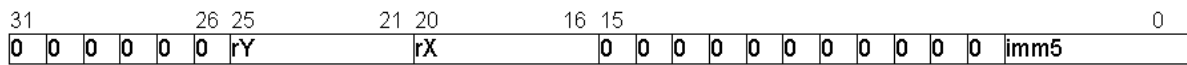


Table 76: srui Instruction Features

Feature	Description
Operation	Shifts right, unsigned (logical)
Description	Shifts the unsigned value in rY right by the number of bits specified by the immediate, storing the result in rX. Available only if the processor was configured with either the MC_BARREL_SHIFT_ENABLED or PL_BARREL_SHIFT_ENABLED option.
Syntax	srui rX, rY, imm5
Example	srui r4, r2, 5
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \gg \text{imm5}$
Result	2 cycles
Issue	1 cycle
See Also	sr, shift right from register; sri, shift right with immediate; sru, shift right, unsigned

sub

Figure 80: sub Instruction

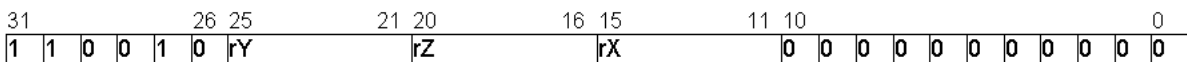


Table 77: sub Instruction Features

Feature	Description
Operation	Integer subtraction
Description	Subtracts the value in rZ from the value in rY, storing the result in rX.
Syntax	sub rX, rY, rZ
Example	sub r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] - \text{gpr}[\text{rZ}]$
Result	1 cycle
Issue	1 cycle
See Also	addi, add with signed immediate

SW

Figure 81: sw Instruction

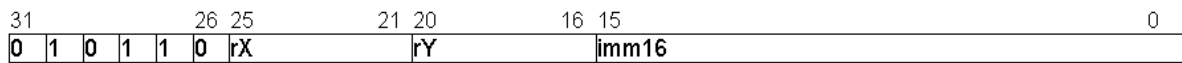


Table 78: sw Instruction Features

Feature	Description
Operation	Store word to memory
Description	Stores the value in rY into memory at the address specified by the sum of the value in rX added to the sign-extended immediate.
Syntax	sw(rX+imm16), rY
Example	sw(r2+8), r4
Semantics	<pre> address = gpr[rX] + sign_extend(imm16) memory[address] = gpr[rY] & 0xff memory[address+1] = (gpr[rY] >> 8) & 0xff memory[address+2] = (gpr[rY] >> 16) & 0xff memory[address+3] = (gpr[rY] >> 32) & 0xff </pre>
Result	
Issue	1 cycle
See Also	sb, store byte; sh, store half-word

wcsr

Figure 82: wcsr Instruction

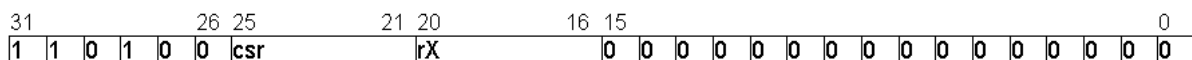


Table 79: wcsr Instruction Features

Feature	Description
Operation	Write control or status register
Description	Writes the value in rX to the specified control or status register.
Syntax	wcsr csr, rX
Example	wcsr IM, r15
Semantics	csr = gpr[rX]
Result	1 cycle
Issue	1 cycle
See Also	rcsr, read control and status register

xnor

Figure 83: xnor Instruction

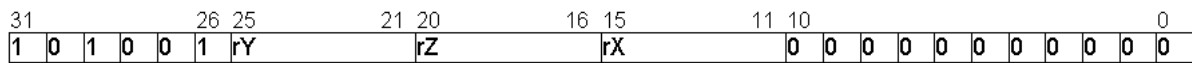


Table 80: xnor Instruction Features

Feature	Description
Operation	Bitwise logical exclusive-NOR
Description	Bitwise exclusive-NOR of the value in rY with the value in rZ, storing the result in rX.
Syntax	xnor rX, rY, rZ
Example	xnor r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \sim(\text{gpr}[\text{rY}] \wedge \text{gpr}[\text{rZ}])$
Result	1 cycle
Issue	1 cycle
See Also	xnori, XNOR with immediate

xnori

Figure 84: xnori Instruction

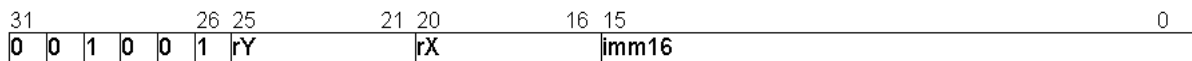


Table 81: xnori Instruction Features

Feature	Description
Operation	Bitwise logical exclusive-NOR
Description	Bitwise exclusive-NOR of the value in rY with the zero-extended immediate, storing the result in rX.
Syntax	xnori rX, rY, imm16
Example	xnori r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \sim(\text{gpr}[\text{rY}] \wedge \text{zero_extend}(\text{imm16}))$
Result	1 cycle
Issue	1 cycle
See Also	xnor, XNOR between registers

xor

Figure 85: xor Instruction

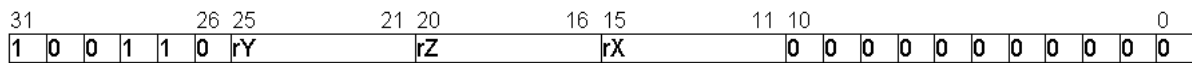


Table 82: xor Instruction Features

Feature	Description
Operation	Bitwise logical exclusive-OR
Description	Bitwise exclusive-OR of the value in rY with the value in rZ, storing the result in rX.
Syntax	xor rX, rY, rZ
Example	xor r14, r15, r17
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \wedge \text{gpr}[\text{rZ}]$
Result	1 cycle
Issue	1 cycle
See Also	xori, XOR with immediate

xori

Figure 86: xori Instruction

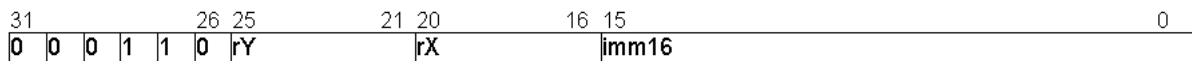


Table 83: xori Instruction Features

Feature	Description
Operation	Bitwise logical exclusive-OR
Description	Bitwise exclusive-OR of the value in rY with the zero-extended immediate, storing the result in rX.
Syntax	xori rX, rY, imm16
Example	xori r4, r2, 0x5555
Semantics	$\text{gpr}[\text{rX}] = \text{gpr}[\text{rY}] \wedge \text{zero_extend}(\text{imm16})$
Result	1 cycle
Issue	1 cycle
See Also	xori, XOR between registers

Index

A

A field **26**
ACK **17**
ACK_I **39**
ACK_O **41**
Acknowledge Input **39**
Acknowledge Output **41**
add instruction **45**
addi instruction **46**
address alignment **14**
Address Input array **40**
Address Output array **39**
Address pipeline stage **5**
address space **13**
ADR_I() **40**
ADR_O() **39**
and instruction **46**
andhi instruction **47**
andi instruction **47**
arithmetic instructions **27**
associativity in caches **15**

B

b instruction **48**
ba register **7**
be instruction **48**
bg instruction **49**
bge instruction **49**
bgeu instruction **50**
bgu instruction **50**
bi instruction **51**
BIE field **9**
big-endian **13**
bne instruction **51**
BP field **11**

BP registers **23, 24, 26**
break instruction **52**
Breakpoint **17, 18, 19, 23**
breakpoint address register **7**
Breakpoint registers **23, 24, 26**
bret instruction **52**
BTE_I() **37**
BTE_O() **37**
Burst Type Extension **37**
bypass in pipeline **5**

C

cache configurations **16, 34**
cacheable addresses **13**
caches **15, 34**
call instruction **53**
calli instruction **53**
CC field **11**
CC register **9, 11**
CFG register **9, 11**
cmpe instruction **54**
cmpei instruction **54**
cmpg instruction **55**
cmpge instruction **56**
cmpgei instruction **56**
cmpgeu instruction **57**
cmpgeui instruction **57**
cmpgi instruction **55**
cmpgu instruction **58**
cmpgui instruction **58**
cmpne instruction **59**
cmpnei instruction **59**
comparison instructions **28**
component signals
 introduction to **38**

- master signals **39**
- slave signals **40**
- configuration options **31**
- Configuration register **9, 11**
- Control and Status Registers
 - Configuration **9, 11**
 - Cycle Counter **9, 11**
 - Data Cache Control **9, 11**
 - Exception Base Address **9, 12, 22**
 - Instruction Cache Control **9, 10**
 - Interrupt Enable **9**
 - Interrupt Mask **9, 10**
 - Interrupt Pending **9, 10**
 - introduction **9**
 - Program Counter **9**
- CR format for instructions **43**
- CTI_I() **36**
- CTI_O() **36**
- CYC_I **41**
- CYC_O **40**
- Cycle Counter **9, 11**
- Cycle Input **41**
- Cycle Output **40**
- Cycle Type Identifier **36**
- CYCLE_COUNTER_ENABLED **32**

D

- D field **11**
- DAT_I() **39, 40**
- DAT_O() **39, 40**
- Data Cache Control **9, 11**
- Data Input array **39, 40**
- Data Output array **39, 40**
- data transfer instructions **28**
- data types **6**
- DataBusError **17, 20**
- DC field **11**
- DC register **24, 25**
- DCACHE_ASSOCIATIVITY **33**
- DCACHE_BASE_ADDRESS **33**
- DCACHE_BYTES_PER_LINE **33**
- DCACHE_ENABLED **33**
- DCACHE_LIMIT **33**
- DCACHE_SETS **33**
- DCC register **9, 11**
- DEBA register **24, 25**
- debug **24, 34**
- Debug Control **24, 25**
- Debug Control and Status Registers **24**
- Debug Exception Base Address **24, 25**
- debug exceptions **18**
- debug monitor **18**
- DEBUG_ENABLED **32, 34**
- Decode pipeline stage **5**
- DIVIDE_ENABLED **31**
- DivideByZero **17, 20**
- divu instruction **60**

E

- E field **26**
- ea register **7**
- EBA register **9, 12, 22**
- EBR **34**
- EIE field **9**
- embedded block ram **34**
- endianness **13**
- eret instruction **60**
- ERR_I **39**
- ERR_O **41**
- Error Input **39**
- Error Output **41**
- exception address register **7**
- Exception Base Address **9, 12, 22**
- exceptions
 - breakpoints **23**
 - debug **18**
 - handlers **18**
 - interrupts **18, 22**
 - introduction to **17**
 - nested **22**
 - non-debug **18**
 - processing **17**
 - reset **23**
 - watchpoints **24**
- Execute pipeline stage **5**
- extended data types **7**

F

- F field in JTAG UART Receive Register **26**
- F field in JTAG UART Transmit Register **26**
- Fetch pipeline stage **5**
- fp register **8, 14**
- frame pointer **8, 14**

G

- G field **11**
- general purpose registers **7**
- global pointer **8**
- gp register **8**

H

- H field **11**
- handlers, exceptions **18**

I

- I bit in Data Cache Control **11**
- I bit in Instruction Cache Control **10**
- I format for instructions **43**
- IC field **11**
- ICACHE_ASSOCIATIVITY **33**
- ICACHE_BASE_ADDRESS **33**
- ICACHE_BYTES_PER_LINE **33**
- ICACHE_ENABLED **32**
- ICACHE_LIMIT **33**
- ICACHE_SETS **33**
- ICC register **9, 10**

- IE field 9
 - IE register 9
 - IM register 9, 10
 - initializing caches 16
 - Instruction Cache Control 9, 10
 - instruction set
 - categories 27
 - descriptions 45
 - add 45
 - addi 46
 - and 46
 - andhi 47
 - andi 47
 - b 48
 - be 48
 - bg 49
 - bge 49
 - bgeu 50
 - bgu 50
 - bi 51
 - bne 51
 - break 52
 - bret 52
 - call 53
 - calli 53
 - cmpe 54
 - cmpei 54
 - cmpg 55
 - cmpge 56
 - cmpgei 56
 - cmpgeu 57
 - cmpgeui 57
 - cmpgi 55
 - cmpgu 58
 - cmpgui 58
 - cmpne 59
 - cmpnei 59
 - divu 60
 - eret 60
 - lb 61
 - lbu 61
 - lhu 62
 - lw 63
 - modu 63
 - mul 64
 - muli 64
 - mv 65
 - mvh 65
 - nor 66
 - nori 66
 - not 67
 - or 67
 - orhi 68
 - ori 68
 - rcsr 69
 - ret 69
 - sb 70
 - scall 70
 - sextb 71
 - sextb 71
 - sh 72
 - sl 72
 - sli 73
 - sr 73
 - sri 74
 - sru 74
 - sruui 75
 - sub 75
 - sw 76
 - wcsr 76
 - xnor 77
 - xnori 77
 - xor 78
 - xori 78
 - formats 43
 - opcodes 44
 - pseudo-instructions 45
 - InstructionBusError 17, 19
 - INT field 11
 - interconnect architecture *see* WISHBONE
 - interconnect
 - interlock in pipeline 5
 - Interrupt 17, 18, 20, 22
 - Interrupt Enable 9
 - Interrupt Mask 9, 10
 - Interrupt Pending 9, 10
 - invalidating caches 16
 - IP register 9, 10
- J**
- J field 11
 - JRX register 24, 26
 - JTAG UART Receive Register 24, 26
 - JTAG UART Transmit Register 24, 26
 - JTX register 24, 26
- L**
- lb instruction 61
 - lbu instruction 61
 - lh instruction 62
 - lhu instruction 62
 - lines in caches 15
 - Lock Input 41
 - Lock Output 40
 - LOCK_I 41
 - LOCK_O 40
 - logic instructions 27
 - lw instruction 63
- M**
- M field 11
 - master signals 39
 - memory architecture
 - address alignment 14
 - address space 13
 - cacheable addresses 13
 - endianness 13

- exceptions 17
- stack layout 14
- Memory pipeline stage 5
- model, programmer's 5
- modu instruction 63
- monitor, debug 64
- mul instruction 64
- muli instruction 64
- mv instruction 65
- mvh instruction 65

N

- nested exceptions 22
- non-debug exceptions 18
- nor instruction 66
- nori instruction 66
- not instruction 67

O

- opcodes 44
- OPENCORES.ORG 35
- or instruction 67
- orhi instruction 68
- ori instruction 68

P

- PC register 9
- pipeline 5, 17
- processing exceptions 17
- Program Counter 9
- program flow control instructions 29
- programmer's model 5
- pseudo-instructions 45

R

- R field 11
- r0 register 7
- ra register 7
- rcsr instruction 69
- read-miss 15
- Registered Feedback mode 36
- registers
 - Control and Status 9
 - Debug Control and Status 24
 - general purpose 7
- Reset 17, 19, 23
- resources 34
- ret instruction 69
- Retry Input 39
- Retry Output 41
- return address register 7
- REV field 11
- RI format for instructions 43
- RR format for instructions 43
- RTY_I 39
- RTY_O 41
- RXD field 26

S

- S field 11
- sb instruction 70
- scall instruction 70
- SEL_I() 40
- SEL_O() 39
- Select Input array 40
- Select Output array 39
- sextb instruction 71
- sexth instruction 71
- sh instruction 72
- shift instructions 28
- SIGN_EXTEND_ENABLED 32
- signals, component 38
- sl instruction 72
- slave signals 40
- sli instruction 73
- SoC Interconnection Architecture for Portable IP Cores see WISHBONE interconnect
- sp register 8, 14
- sr instruction 73
- sri instruction 74
- sru instruction 74
- srui instruction 75
- stack layout 14
- stack pointer 8, 14
- stages in pipeline 5
- STB_I 41
- STB_O 40
- Strobe Input 41
- Strobe Output 40
- sub instruction 75
- sw instruction 76
- SystemCall 17, 20
- System-on-Chip Interconnection Architecture for Portable IP Cores see WISHBONE interconnect

T

- TXD field 26

W

- Watchpoint 17, 18, 20, 24
- Watchpoint registers 24, 27
- ways in caches 15
- wcsr instruction 76
- WE_I 40
- WE_O 39
- WISHBONE interconnect 35
 - component signals 38
 - introduction to 35
 - master signals 39
 - Registered Feedback mode 36
 - slave signals 40
- WP field 11
- WP registers 24, 27
- Write Enable Input 40
- Write Enable Output 39

Writeback pipeline stage **5**

X

X field **11**

xnor instruction **77**

xnori instruction **77**

xor instruction **78**

xori instruction **78**